

Republic of Tunisia
Ministry of Higher Education
and Scientific Research

University of Sfax
National School of Electronics
and Telecommunications of Sfax



Engineer in :
Data Engineering and Decisional Systems

Option :
Data Engineering

Ordre Nb: IDSD-ID-3-2026-[TBD]

Dissertation

Presented at

National School of Electronics and Telecommunications of Sfax

in order to obtain the

**National Engineering Diploma in :
Data Engineering and Decisional Systems**

**Option:
Data Engineering**

by

Mohamed GUIDARA

**Design and Implementation of Itqan:
An AI-Enabled RFQ Lifecycle Intelligence Platform for
Industrial Estimation Process Optimization**

Presented on: 24/06/2026, before the jury of commission:

Mr./Mrs. [TBD]	President
Mr./Mrs. [TBD]	Academic Supervisor
Mr./Mrs. [TBD]	Professional Supervisor
Mr./Mrs. [TBD]	Reviewer



DEDICATION

To Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam.

To my dear, First name.

To Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi.

To my dear, First name.

To

To all of you,

I dedicate this work.

Mohamed GUIDARA



Acknowledgement

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue. Ut in risus volutpat libero pharetra tempor. Cras vestibulum bibendum augue. Praesent egestas leo in pede. Praesent blandit odio eu enim. Pellentesque sed dui ut augue blandit sodales. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam nibh. Mauris ac mauris sed pede pellentesque fermentum. Maecenas adipiscing ante non diam sodales hendrerit.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue. Ut in risus volutpat libero pharetra tempor. Cras vestibulum bibendum augue. Praesent egestas leo in pede. Praesent blandit odio eu enim. Pellentesque sed dui ut augue blandit sodales. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam nibh. Mauris ac mauris sed pede pellentesque fermentum. Maecenas adipiscing ante non diam sodales hendrerit.



TABLE OF CONTENTS

LIST OF FIGURES	vi
LIST OF TABLES	vii
LIST OF ABBREVIATIONS	viii
GENERAL INTRODUCTION	1
1 General Context and Business Problem	2
1.1 INTRODUCTION	3
1.2 Host Organization and Industrial Context	3
1.3 The RFQ Process in Industrial Manufacturing	5
1.4 Audit Findings and Current Process Limitations	7
1.5 From BOQ Automation to Lifecycle Intelligence: A Strategic Pivot	12
1.6 The Itqan Vision: An RFQ Lifecycle Intelligence Platform	15
1.7 Project Methodology Overview	18
1.8 CONCLUSION	18
2 Requirements, Methodology, and Project Management	20
2.1 INTRODUCTION	21
2.2 Stakeholders and Actors	21
2.3 Functional Requirements	23
2.4 Non-Functional Requirements	28
2.5 Product Backlog	31
2.6 Use Cases and Activity Diagrams	35
2.7 Methodology and Project Management	38
2.8 Traceability Matrix	42
2.9 CONCLUSION	44
3 Conceptual and Technical Architecture	45
3.1 INTRODUCTION	46

3.2	Itqan Global Architecture and Four-Pillar Vision	46
3.3	Microservices Architecture	50
3.4	RFQ Manager Service: Lifecycle Orchestration Backbone	53
3.5	RFQ Copilot Service: Trust-Bound Conversational Architecture	58
3.6	Evidence Boundary and Source-of-Truth Strategy	67
3.7	Data Model and API Contracts	71
3.8	Platform Integration: UI and Intelligence Layers	74
3.9	Architectural Decisions and Trade-offs	76
3.10	CONCLUSION	78
4	Implementation	80
4.1	INTRODUCTION	81
4.2	Development Environment and Toolchain	81
4.3	Implementation of <code>rfq_manager_ms</code>	86
4.3.1	BACAB Layered Structure in Code	86
4.3.2	RFQ Lifecycle and Stage Progression	88
4.3.3	Subtasks, Notes, Files, and Reminders	89
4.3.4	Statistics and Analytics Endpoints	90
4.3.5	Containerization and Hardening	91
4.4	Implementation of <code>rfq_copilot_ms</code>	92
4.4.1	Turn Processing Pipeline and First Vertical Slice	93
4.4.2	Path Registry and Planning Logic	96
4.4.3	ExecutionPlanFactory and Single-Construction Discipline	97
4.4.4	Memory, Guardrails, and Judge Layer	98
4.4.5	Escalation Gate and Safe Failure Modes	99
4.5	Platform Integration: UI and Intelligence Modules	101
4.6	CONCLUSION	106
5	Validation, Results, and Discussion	107
5.1	INTRODUCTION	108
5.2	Backend Validation of <code>rfq_manager_ms</code>	110
5.2.1	Endpoint Coverage and Response Validation	111
5.2.2	Workflow and Data Consistency	112
5.3	Copilot Validation: Formal Metrics	115
5.4	Copilot Validation: Golden Scenarios	118
5.5	Integration Validation	119

TABLE OF CONTENTS

5.5.1	Integration Paths and Validation Status	120
5.5.2	Demonstration-Oriented Evidence	122
5.5.3	Intelligence Service Validation in the Integrated Context	122
5.5.4	Summary of Integration Validation	123
5.6	Business Impact Against Audit Pain Points	124
5.7	Limitations and Honest Discussion	127
5.8	Future Work	129
5.9	CONCLUSION	131
GENERAL CONCLUSION		133
BIBLIOGRAPHY		134
APPENDICES		137
A.1	Detailed Use Case Descriptions	137
A.2	Additional Sequence Diagrams	137
A.3	Selected Path Registry Excerpt	137
A.4	Copilot Prompt Excerpt and Response Rules	138
A.5	Golden Scenario Test Set	138
A.6	API and Postman Validation Structure	138

LIST OF FIGURES

1.1	Current RFQ process flow at GHI, organized in six phases across the departments involved	7
1.2	Pain points and operational consequences mapped to the RFQ lifecycle phases .	9
1.3	The four-pillar target architecture of Itqan and its cross-cutting conversational layer	16
2.1	System-level use case diagram of the Itqan platform	36
2.2	Activity diagram for the <i>Advance RFQ Stage</i> use case	37
2.3	Activity diagram for the <i>Ask Copilot About an RFQ</i> use case	38
2.4	Consolidated sprint timeline of the project, showing iteration boundaries and the distribution of work across the platform's services	40
3.1	Global architecture of the Itqan platform: four layers and the cross-cutting conversational layer	47
3.2	Microservices architecture of the Itqan platform, showing service boundaries and inter-service communication	51
3.3	Entity-relationship diagram of the manager service domain model	55
3.4	Trust-boundary architecture of the RFQ Copilot, with components colored by trust class	62
3.5	The evidence boundary of the RFQ Copilot: decision flow from turn classification to evidence source selection	70
3.6	Platform integration view: the end-to-end request flow across the four implemented services	76
4.1	Folder structure of the <code>rfq_manager_ms</code> service realizing the BACAB layered pattern	87
4.2	Turn processing pipeline of <code>rfq_copilot_ms</code> (v2 implementation), with deterministic stages distinguished from language-model invocations	93
4.3	Representative screen of <code>rfq_ui_ms</code> : RFQ detail view with the copilot drawer open in RFQ-bound mode	104



LIST OF TABLES

1.1	The twelve documented pain points, grouped by structural family	11
2.1	Stakeholder alignment matrix showing the cadence, alignment approach, and proof artifacts for each project stakeholder	22
2.2	Functional requirements of the manager service	25
2.3	Functional requirements of the copilot service	26
2.4	Functional requirements of the intelligence service	27
2.5	User interface and cross-cutting functional requirements	28
2.6	Non-functional requirements grouped by category	30
2.7	Manager service backlog	32
2.8	Copilot service backlog	33
2.9	Intelligence service and cross-cutting backlog	34
2.10	Traceability matrix linking functional requirements to implementing modules and to the validation evidence in Chapter 5	43
3.1	Scope status of the four pillars at the conclusion of the present work	49
3.2	The eight trust boundaries of the copilot architecture and their owners	61
3.3	Architectural decisions register summary	77
4.1	Principal technologies used in the implementation of the Itqan platform	85
5.1	Consolidated summary of validation evidence assembled across the chapter	110
5.2	Summary of validation evidence for the manager service	114
5.3	Breakdown of the copilot service pytest suite by category	117
5.4	Integration paths and their validation status in the present work	121
5.5	Business impact mapping: audit pain points against implemented capabilities	125



LIST OF ABBREVIATIONS

ADR Architectural Decision Record

API Application Programming Interface

ASME American Society of Mechanical Engineers

BACAB BACAB Consulting layered architecture pattern

BI Business Intelligence

BOQ Bill of Quantities

CI Continuous Integration

CRUD Create, Read, Update, Delete

ERD Entity-Relationship Diagram

GHI Gulf Heavy Industries

HTTP HyperText Transfer Protocol

IAM Identity and Access Management

IDSD Data Engineering and Decisional Systems

JSON JavaScript Object Notation

KPI Key Performance Indicator

LLM Large Language Model

MR Material Requisition

LIST OF ABBREVIATIONS

MS Microservice

PFE Projet de Fin d'Études (Final Year Project)

QC Quality Control

RAG Retrieval-Augmented Generation

REST Representational State Transfer

RFQ Request for Quotation

RVL Restricted Vendor List

SAES Saudi Aramco Engineering Standard

SAMSS Saudi Aramco Materials System Specification

SQL Structured Query Language

TEMA Tubular Exchanger Manufacturers Association

UI User Interface

UML Unified Modeling Language



GENERAL INTRODUCTION

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue. Ut in risus volutpat libero pharetra tempor. Cras vestibulum bibendum augue. Praesent egestas leo in pede. Praesent blandit odio eu enim. Pellentesque sed dui ut augue blandit sodales. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam nibh. Mauris ac mauris sed pede pellentesque fermentum. Maecenas adipiscing ante non diam sodales hendrerit.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue. Ut in risus volutpat libero pharetra tempor. Cras vestibulum bibendum augue. Praesent egestas leo in pede. Praesent blandit odio eu enim. Pellentesque sed dui ut augue blandit sodales. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam nibh. Mauris ac mauris sed pede pellentesque fermentum. Maecenas adipiscing ante non diam sodales hendrerit.

General Context and Business Problem

Contents

1.1	INTRODUCTION	3
1.2	Host Organization and Industrial Context	3
1.3	The RFQ Process in Industrial Manufacturing	5
1.4	Audit Findings and Current Process Limitations	7
1.5	From BOQ Automation to Lifecycle Intelligence: A Strategic Pivot	12
1.6	The Itqan Vision: An RFQ Lifecycle Intelligence Platform .	15
1.7	Project Methodology Overview	18
1.8	CONCLUSION	18

1.1 INTRODUCTION

A request for quotation (RFQ) in industrial manufacturing is rarely a simple commercial document. It is the entry point of a long, multi-departmental process that mobilizes engineering, production, quality, and procurement expertise over weeks or months, and whose outcome can determine whether a project is won, lost, or accepted under unfavorable terms. The challenge addressed by this work is not the production of quotations themselves, but the structural fragility of the process that surrounds them.

This chapter establishes the context that motivates Itqan, the platform proposed and developed in this project. It begins by introducing the host organization, BACAB Consulting, and the industrial client, GHI / Albassam Group, situating the work within its operational and academic context. It then describes the request for quotation process as it exists in industrial manufacturing, with attention to the artifacts and standards that distinguish this domain from generic commercial estimation. The chapter continues with the audit findings that revealed the structural pain points of the current process, and recounts the strategic pivot from automating the Bill of Quantities to building a platform for RFQ lifecycle intelligence. It closes by introducing the Itqan vision and the four-pillar target architecture, and by giving a brief overview of the project methodology that is developed in detail in the following chapter.

1.2 Host Organization and Industrial Context

This work was conducted as a final-year engineering internship at BACAB Consulting, a Tunisia-based consulting firm specializing in cloud architecture, microservices, artificial intelligence, and data engineering. Headquartered in Sfax, the firm operates as a small but specialized team that combines strategic advisory with hands-on technical delivery, positioning itself as a partner for organizations undertaking digital and analytical transformation. The internship took place within this team and was organized around a single client engagement that became the practical anchor of the present project.

The client is GHI / Albassam Group, a major industrial conglomerate based in Dammam, Kingdom of Saudi Arabia. The Al Bassam Group is one of the older private conglomerates in the Kingdom, with origins in petroleum equipment trading and present-day activities spanning industrial manufacturing, inspection and testing, and a range of related services. Within this group, the engagement focused on Gulf Heavy Industries Co., commonly referred to as GHI, a wholly owned subsidiary established in 1984 and recognized as one of the largest specialized heavy steel fabrication companies in the Middle East. GHI operates two manufacturing facilities in Jubail Industrial City and produces pressure vessels, heat exchangers, and related heavy industrial equipment for customers in the oil and gas, petrochemical, and energy sectors, with Saudi Aramco among its principal clients.

The engagement with the Al Bassam Group was a new commercial relationship for BACAB Consulting, secured through a presales and negotiation phase that began in November and culminated in a five-day on-site audit conducted between Jubail and Dammam. The audit followed a top-down structure that started at the group level, with sessions involving the chairman, the chief executive, and human resources at the Dammam headquarters, before shifting to the operational level at the GHI manufacturing facility. There, the audit team engaged primarily with the estimation department and its full set of roles, with the general manager of GHI, and with selected finance and engineering staff whose work intersects the estimation process. The active development phase of the project began at the start of the following January, after the audit findings had been consolidated and a refined scope agreed upon.

Industrial estimation in this context differs in several important ways from generic commercial estimation. The artifacts that drive a quotation are heavy in domain expertise: pressure vessel and heat exchanger fabrication is governed by a layered set of standards, requires the involvement of multiple engineering disciplines before a price can be defended, and exposes the manufacturer to long lead times and significant contractual liabilities once a bid is accepted. These characteristics make the surrounding process, rather than the calculation of the price itself, the primary source of operational risk. Understanding this distinction was essential to framing the project, and is developed further in the next section.

1.3 The RFQ Process in Industrial Manufacturing

A Request for Quotation, abbreviated RFQ, is the formal mechanism by which an industrial buyer asks a manufacturer to price a defined scope of work. In the context of pressure vessel and heat exchanger fabrication, the RFQ is rarely a simple price request: it arrives as a multi-document package whose interpretation, costing, and approval mobilize several departments over weeks or months, and whose accuracy carries significant contractual and financial consequences once a bid is submitted. This section presents the RFQ as a domain object, the artifacts that surround it, and the end-to-end estimation workflow as observed at GHI during the on-site audit.

The RFQ Package and Its Surrounding Artifacts

When an RFQ is received at GHI, it is rarely a single document. The most consequential RFQs from major clients arrive as a structured package built around a Material Requisition, abbreviated MR, which carries the canonical reference number used throughout the project. A representative MR package observed during the audit, identified by the reference SA-AYPP-6-MR-022, contained around fifty-five business files organized in sixteen numbered sections covering scope of supply, bill of materials, restricted vendor lists, technical specifications, drawings, applicable standards, and quality assurance and inspection requirements.

Three artifacts inside the package drive most of the estimation work. The Bill of Materials, abbreviated BOM, is a spreadsheet listing every line item to be fabricated, with tag numbers, design codes such as *ASME Section VIII Division 1*, services such as *collection vessel*, dimensions, materials, and quantities. The Restricted Vendor List, abbreviated RVL, enumerates the vendors authorized by the client to supply specific materials, and constrains procurement decisions later in the lifecycle. The third class of artifact is the body of applicable standards, organized at GHI under named families: SAMSS for materials, SAES for engineering specifications, SAEP for engineering procedures, and SA-175 forms for compliance. International standards such as those issued by the American Society of Mechanical Engineers (ASME) and the Tubular

Exchanger Manufacturers Association (TEMA) define the codified rules for pressure vessel and heat exchanger fabrication and are referenced throughout the package.

The Estimation Workbook

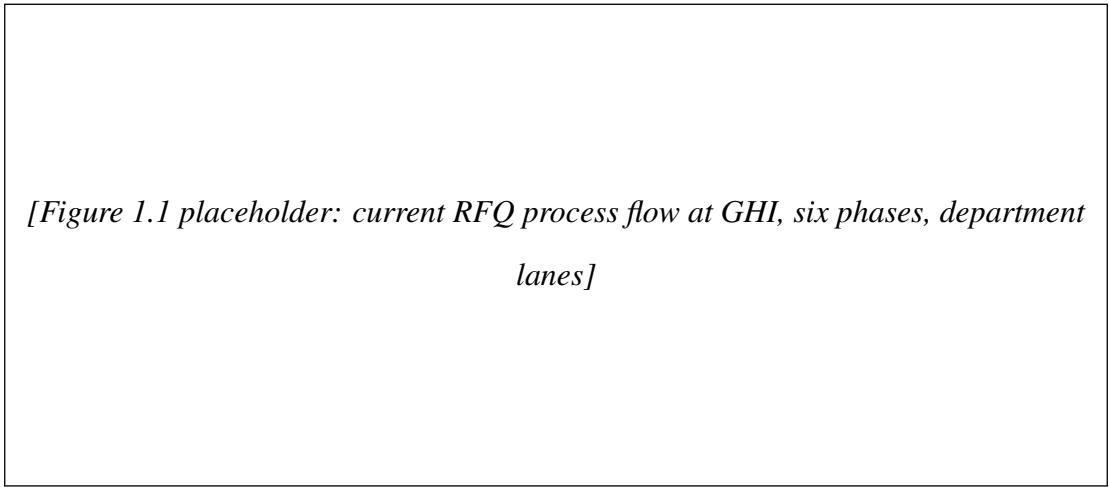
Once the RFQ package has been received and triaged, the estimation team produces a quotation by building an estimation workbook. The workbook is a single Microsoft Excel file structured around a fixed template of thirty-six interconnected sheets. It captures every cost driver involved in fabricating the scope: material breakdown, direct and indirect labour and man-hours, fabrication and design hours, consumables, surface preparation and paint, non-destructive testing, post-weld heat treatment, ASME and U-Stamp certification costs, transportation, insurance, financial charges, contingency, and overheads. Several sheets aggregate these inputs into a Bid Summary and a Top Sheet that present the bid to internal approvers and, after revision, to the client. Each line is expressed simultaneously in Saudi Riyals, US Dollars, percentage share, and cost per ton, reflecting the multi-currency and weight-driven nature of heavy steel fabrication.

The workbook is filled almost entirely by senior estimators whose expertise is not codified in any external document. The same template is used for every RFQ; what differs is the judgment encoded into its cells. This characteristic is central to understanding both the audit findings and the strategic pivot presented in the following sections.

The End-to-End Estimation Workflow

The RFQ lifecycle as observed at GHI follows six broad phases. The first phase is intake and triage: the RFQ arrives by direct email from the client or through the SAP Ariba procurement portal, and the department manager performs a brief feasibility check before deciding whether to register the RFQ in the bidding list. Approximately ninety percent of received RFQs pass this initial filter. The second phase is registration and classification: the RFQ is assigned a unique reference number and classified as either Internal Bid or Internal Fabrication, a distinction that determines downstream workflow. The third phase is design: depending on whether existing design data is reusable, the engineering team either reuses prior designs or produces a basic

design from scratch, after which the detailed BOM is consolidated. The fourth phase is cost estimation: the workbook is filled across its thirty-six sheets, drawing on material pricing, labour estimates, and standards-driven cost components. The fifth phase is the fabrication decision, where the team determines whether the work will be executed internally or subcontracted. The sixth and final phase is submission and follow-up: the bid is approved internally, transmitted to the client, and tracked through one of three terminal outcomes (won, lost, pending) with manual follow-up in the third case. Figure 1.1 presents the full lifecycle as observed during the audit.



[Figure 1.1 placeholder: current RFQ process flow at GHI, six phases, department lanes]

Figure 1.1: Current RFQ process flow at GHI, organized in six phases across the departments involved

This workflow involves four to five distinct departments at GHI: Estimation as the central coordinator, Engineering for design feasibility, Production for fabrication input, Quality Control for inspection requirements, and Procurement for material and vendor decisions. The fragility introduced by this multi-departmental coordination, together with the multi-week timeline of each RFQ, is precisely what motivated the audit and is the subject of the next section.

1.4 Audit Findings and Current Process Limitations

The audit conducted at GHI was not merely a meeting series. It was a deliberate fact-finding exercise designed to surface the structural and operational limitations of the existing RFQ process before any technical proposal was formulated. This section presents the audit methodology,

the pain points it identified grouped by structural family, and a complete reference of the twelve documented pain points.

Audit Methodology

The audit was conducted over five days in the second half of November, alternating between the Al Bassam Group headquarters in Dammam and the GHI manufacturing facility in Jubail Industrial City. The sessions at the Dammam headquarters engaged senior leadership of the group, including the chairman, the chief executive, and the human resources function, in order to establish strategic context and align on the scope of the engagement. The sessions at the GHI facility shifted to the operational level: in addition to the General Manager of GHI, the audit team engaged the entire estimation department across all of its roles, alongside selected representatives from the engineering and finance departments whose responsibilities intersect the estimation process.

The methodology combined three complementary inputs: structured interviews focused on responsibilities, daily workflows, and recurring frustrations; direct observation of the process artifacts in use, including the estimation workbook and the documents that surround a typical RFQ; and a review of past projects whose outcomes illustrated where the process tended to break. Findings derived directly from on-site observation are referred to as audit findings, and findings that emerged later during the analytical review of the workbook structure and workflow logic are referred to as analytical findings. Both categories are presented together below because they describe the same operational reality from complementary angles. Figure 1.2 visualizes how each family of pain points manifests across the lifecycle phases of Figure 1.1.

[Figure 1.2 placeholder: pain points mapped to RFQ lifecycle phases with cost and delay annotations]

Figure 1.2: Pain points and operational consequences mapped to the RFQ lifecycle phases

Pain Points Grouped by Family

Read structurally, the twelve pain points cluster into five families that share a common underlying weakness. The first family, **coordination and communication**, captures the absence of cross-departmental status tracking (PP-01) and the manual nature of all client interaction (PP-02): both manifest as information that fails to reach the right person at the right time. The second family, **visibility and reporting**, captures the absence of any executive-level view of the pipeline (PP-03) and the intuition-based prioritization of bids (PP-11): leadership cannot answer basic questions about the state or quality of the estimation portfolio. The third family, **historical learning**, captures the systematic non-reuse of past RFQs (PP-04) and the loss of feasibility expertise in undocumented gut-checks (PP-07): institutional memory does not feed forward into future work. The fourth family, **feasibility and supplier risk**, captures the late discovery of project-killing blockers (PP-05) and the unmanaged supplier exposure introduced by client-mandated vendor lists (PP-09): risk surfaces too late to act on it. The fifth family, **tooling fragility**, captures the structural brittleness of the estimation workbook as a single point of failure (PP-06), the underutilization of the deployed ERP system (PP-10), the absence of classification-driven workflow enforcement (PP-08), and the late recognition of compliance-driven cost implications (PP-12): the supporting tools do not enforce the discipline that the process needs.

Reference: The Twelve Documented Pain Points

Table 1.1 provides the complete reference of the twelve pain points, with the family to which each belongs, a short title, and a concise statement of the operational consequence. This table is referenced again in Chapter 5, where each Itqan capability is mapped back to the pain points it addresses.

ID	Family	Title	Consequence
PP-01	Coordination	Inter-departmental communication breakdown	Information failures cause lost projects across silos.
PP-02	Coordination	Manual client follow-up	Missed deadlines and lost projects from manual tracking.
PP-03	Visibility	Zero executive visibility	Leadership has no real-time view of the estimation pipeline.
PP-04	Historical learning	No reuse of past RFQs	Each RFQ treated as a fresh exercise; institutional memory wasted.
PP-05	Feasibility risk	Late discovery of infeasibility	Project-killing blockers found after significant work invested.
PP-06	Tooling fragility	Workbook as single point of failure	Critical financial model with no version control or concurrent edit.
PP-07	Historical learning	Undocumented feasibility expertise	Manager go/no-go reasoning lost; cannot transfer or analyze.
PP-08	Tooling fragility	Classification not enforcing workflow	IB/IF classification not driving downstream notifications or gates.
PP-09	Feasibility risk	Unmanaged supplier risk	No tracking of supplier reliability, lead times, or pricing trends.
PP-10	Tooling fragility	Underutilized ERP system	Odoo deployed but not integrated with estimation workflow.
PP-11	Visibility	Intuition-based bid prioritization	Focused-bid decisions made without win-probability analysis.
PP-12	Tooling fragility	Standards not driving early cost implications	ASME/TEMA/FATI implications recognized too late in estimation.

From Symptoms to Diagnosis

These twelve pain points are not independent problems to be solved one at a time. Read together, they describe a single underlying condition: the absence of a structured, observable, and traceable representation of the RFQ lifecycle that the entire organization can act on. The estimation expertise embodied in the workbook is mature, the standards are well known, and the staff are competent. The standards are well known, and the staff are competent. What is missing is the connective tissue that turns a sequence of expert acts into a coordinated process. This diagnosis is the foundation for the strategic reframing presented in the next section.

1.5 From BOQ Automation to Lifecycle Intelligence: A Strategic Pivot

The original technical request from GHI, transmitted to BACAB Consulting before the audit, was specific and ambitious: to design and implement an artificial intelligence system capable of automating the production of the Bill of Quantities (BOQ) and, by extension, the cost estimation that the workbook produces. The premise was that recent advances in large language models and structured data extraction had reached a maturity sufficient to encode estimation expertise into a generative system. The audit conducted in November and the analytical review that followed examined this premise rigorously and concluded that, as stated, it was not feasible. This section explains the basis of that conclusion and presents the reframing that became the foundation of the project.

Why BOQ Automation Was Not the Right Target

Three independent observations led to the conclusion that BOQ automation could not be delivered as a credible system within the scope of this project, or indeed as a credible system at all without a substantially different starting point.

The first observation concerns the nature of the expertise embedded in the workbook. Each of the thirty-six sheets is filled by senior estimators who hold many years of experience in pressure vessel and heat exchanger fabrication. The values they enter reflect implicit reasoning about material behavior under specific operating conditions, supplier reliability in the current market, the expected difficulty of fabrication for a given geometry, and the contractual posture appropriate for a particular client. This reasoning is not documented anywhere outside the estimators' minds. It is also not extractable from the workbook itself: the workbook is the output of estimation, not its specification.

The second observation concerns the absence of a usable historical dataset. A serious attempt at automating cost estimation would require thousands of historical RFQs labeled with their final outcomes, paired with the reasoning that produced their prices. GHI maintains many years of past workbooks, but these are stored as individual Excel files without structured metadata, without outcome labels, and without the contextual notes that would make them suitable as training data. The historical record, as it exists today, can serve as reference material for human estimators but cannot serve as ground truth for a learning system.

The third observation concerns the asymmetric cost of error. In a quotation for a multi-million Saudi Riyal project, an underestimated price commits GHI to a contractual loss that may exceed the entire project margin, while an overestimated price loses the bid to a competitor. There is no acceptable failure mode for an automated system in this regime, and no plausible path from a current state of zero structured historical data to a model whose error bars would be tolerable. Even an advisory system, deployed without authority over the final price, would face high resistance from estimators whose accountability remains unchanged whether or not the system is consulted.

The Reframing

Rather than weakening the original goal into a partial BOQ assistant whose value would be marginal and whose risk would not be, the audit findings suggested a different orientation. The pain points presented in Section 1.4 are not pain points of price calculation. They are pain

points of coordination across departments, of visibility for leadership, of historical learning lost between projects, of late discovery of infeasibility, of fragile tooling, and of communication tracked only in scattered email threads. The estimation workbook itself, despite its fragility as a single point of failure, is not the bottleneck of the process. The bottleneck is everything around it.

The reframing was therefore stated explicitly: rather than attempt to automate the part of the process that depends on irreducible human expertise, build a platform that strengthens every other part of the process and that respects the expertise it cannot replace. Such a platform would track the full lifecycle of an RFQ across departments, give leadership the visibility it lacks, capture and structure the feasibility decisions and outcomes that are currently lost, surface infeasibility risks earlier, and provide a conversational interface for the people who interact with these artifacts daily. The price calculation would remain where it belongs, in the hands of senior estimators using a tool they understand. The platform would surround that core, not replace it.

From Reframing to Project Definition

This pivot is consequential beyond the immediate project. It changes the kind of system being built, the kind of evidence required to validate it, and the kind of value it delivers to the client. A BOQ automator would be a closed-loop generative system whose worth is measured against estimator output. A lifecycle intelligence platform is an open-loop coordination system whose worth is measured against the duration, traceability, and decision quality of the RFQ process as a whole. The two systems have almost nothing in common architecturally, evaluatively, or commercially.

The remainder of this chapter introduces the platform that resulted from this reframing, which has been given the working name Itqan, and presents the four-pillar vision and the methodology that structured its development.

1.6 The Itqan Vision: An RFQ Lifecycle Intelligence Platform

The reframing presented in the previous section called for a platform that surrounds the estimation workbook rather than replaces it. This section introduces that platform under its working name, Itqan, presents the four-pillar target architecture that defines its long-term scope, and clearly separates what has been implemented in the present work from what remains as future development.

Project Identity

The name *Itqan* is drawn from the Arabic word *itqān*, which denotes mastery, precision, and the act of doing work properly and to completion. The choice reflects the spirit of the project: in an industrial estimation context where errors carry significant contractual consequences, the goal of the platform is not to accelerate the process at the expense of rigor, but to bring discipline, traceability, and structured intelligence to a workflow that has historically relied on individual diligence to compensate for the absence of system-level support. Itqan is therefore positioned as an instrument of process discipline, not an instrument of automation in the pejorative sense.

The Four-Pillar Target Architecture

The diagnostic in Section 1.4 identified twelve pain points spanning communication, visibility, decision support, knowledge retention, and tooling. Read structurally, these pain points cluster into four areas of concern, and the target architecture of Itqan is organized around the same four areas. Each pillar addresses a coherent group of pain points and corresponds to a distinct technical capability within the platform. Figure 1.3 presents the four-pillar architecture and the cross-cutting copilot layer.

[Figure 1.3 placeholder: Itqan four-pillar target architecture with cross-cutting copilot layer]

Figure 1.3: The four-pillar target architecture of Itqan and its cross-cutting conversational layer

The first pillar, **Workflow and Tracking**, provides the operational backbone of the platform. It maintains a structured representation of every RFQ across its full lifecycle, tracks the progress of each stage across the departments involved, captures the artifacts and decisions accumulated along the way, and gives every role its appropriate view of the system. This pillar directly addresses PP-01, PP-05, PP-06, PP-08, and contributes to PP-07.

The second pillar, **Communication Automation**, addresses the manual nature of client interaction that has historically caused missed deadlines and lost projects. Its target capabilities include the unified monitoring of inbound channels (direct email and the SAP Ariba portal), assisted drafting of follow-up correspondence with human-in-the-loop approval before any outbound action, deadline tracking with escalation alerts, and a complete communication audit trail tied to each RFQ. This pillar addresses PP-02 directly.

The third pillar, **Executive Intelligence and Business Intelligence**, gives leadership the real-time visibility into the estimation pipeline that is currently absent. Its target capabilities include key performance indicator dashboards covering pipeline value, win rate, client profitability, and department bottlenecks, along with analytical views that support strategic decisions about which clients and project types deserve focused effort. This pillar addresses PP-03 and PP-11.

The fourth pillar, **Historical Learning and Prediction**, is the longest-horizon pillar of the platform. It targets the systematic exploitation of past RFQs to inform future ones: pattern matching against similar historical projects, preliminary cost-range guidance derived from comparable

bids, supplier intelligence built from the cumulative procurement history, and, in the longer term, win-probability estimation conditioned on client and project characteristics. This pillar addresses PP-04, PP-09, PP-10, PP-12, and reinforces PP-07.

The Cross-Cutting Intelligence Layer

The four pillars define the functional surface of the platform, but they do not by themselves describe how a user reaches the information the platform contains. Itqan therefore includes a conversational layer, the RFQ Copilot, that allows authorized users to query the platform in natural language, with strict guarantees about the evidence on which its answers are grounded. The copilot does not own any pillar of its own; it navigates and explains the data that the pillars produce. The architecture and innovation contributions of the copilot are presented in detail in Chapter 3.

Scope of the Present Work

The four-pillar architecture defines the long-term scope of the platform, not the scope delivered by the present work. A clear separation between the implemented scope and the future scope is essential to the honest presentation of this project.

Within the scope of the present work, the first pillar is substantially implemented through the operational backbone service `rfq_manager_ms`, which provides RFQ registration and tracking, workflow-driven stage progression, supporting resources for subtasks, notes, files, and reminders, and the analytical endpoints that feed dashboards. The conversational layer is implemented through `rfq_copilot_ms`, which represents the principal innovation contribution of the project and is presented in depth in subsequent chapters. The remaining capabilities of the four pillars are present at varying levels of maturity: parts of the second and third pillars are exercised by the user interface and the manager service in demonstration form, while the fourth pillar is represented by a foundation of document parsing capabilities in the intelligence service, with the predictive components left as future work.

The next chapter formalizes the requirements that frame the implemented scope, the methodology that structured its development, and the project management practices that supported it.

1.7 Project Methodology Overview

The methodology adopted for the development of Itqan is Scrumban, a hybrid of Scrum and Kanban that retains the iterative cadence and demo-oriented discipline of Scrum while substituting Kanban-style work-in-progress limits for the rigidity of fixed-length sprints. This choice fits the realities of a solo project better than pure Scrum: the work-in-progress discipline of Kanban keeps the developer from spreading attention thinly across several platform microservices, while the sprint cadence preserves the demo and review rhythm that anchors stakeholder communication. The Scrumban board used throughout the project follows a six-state workflow that culminates in a *Documented* terminal state, treating documentation as part of the definition of done rather than as an afterthought.

Chapter 2 presents the requirements, methodology, project management practices, and supporting disciplines (architectural decision tracking and risk management) in full.

1.8 CONCLUSION

This chapter has established the context that motivates Itqan. It introduced BACAB Consulting as the host of this end-of-studies project and GHI / Albassam Group as the industrial client whose RFQ process became its subject. It described the request for quotation lifecycle as it operates in industrial pressure vessel and heat exchanger manufacturing, with the artifacts and standards that distinguish this domain from generic commercial estimation. It presented the audit findings that surfaced twelve recurrent pain points across communication, visibility, decision support, knowledge retention, and tooling. It recounted the strategic pivot from an initial request for BOQ automation toward a platform for RFQ lifecycle intelligence, and explained the basis on which that reframing was made. It introduced the Itqan platform under its working name, presented the four-pillar target architecture that defines its long-term scope, and gave a

brief overview of the methodology that structured its development. The next chapter formalizes the requirements, methodology, and project management practices that disciplined the work.

Requirements, Methodology, and Project Management

Contents

2.1	INTRODUCTION	21
2.2	Stakeholders and Actors	21
2.3	Functional Requirements	23
2.4	Non-Functional Requirements	28
2.5	Product Backlog	31
2.6	Use Cases and Activity Diagrams	35
2.7	Methodology and Project Management	38
2.8	Traceability Matrix	42
2.9	CONCLUSION	44

2.1 INTRODUCTION

This chapter formalizes the engineering structure of the project. It begins by identifying the stakeholders and actors that shape the system’s requirements, then presents the functional and non-functional requirements derived from the audit findings of the previous chapter. The chapter continues with the product backlog, the use case decomposition, and the activity diagrams that capture the most consequential workflows. It then presents the project management practices that disciplined the work, including the choice of Scrumban as the iteration framework, the architectural decision register, and the risk management practice. The chapter closes with a traceability matrix that links each requirement to the module that implements it and to the validation evidence that supports it in Chapter 5.

2.2 Stakeholders and Actors

The system serves and answers to two distinct populations. The first is the population of *stakeholders*: the people whose expectations frame the project’s definition of success but who do not necessarily interact with the platform in its operational form. The second is the population of *actors*: the roles that interact with Itqan through its user interfaces, its API, or its conversational layer. Both populations must be characterized explicitly because they shape requirements in different ways.

Project Stakeholders

Four stakeholders shape the success criteria of this project. The **academic supervisor** at ENET’COM ensures the work meets the methodological and reporting standards of an end-of-studies project, and is the primary audience for early report drafts and architectural decision reviews. The **professional supervisor** at BACAB Consulting ensures the work delivers value to the firm and to its client, and is the primary audience for weekly progress updates and demo-ready milestones. The **technical and leadership stakeholders at GHI / Albassam Group** ensure the

implemented capabilities reflect the operational reality observed during the audit, and are the primary audience for incremental functional demos. The **academic jury** evaluates the project at its defense and is the audience for the final report, the defense presentation, and the validation evidence that supports both. Each stakeholder operates on a different cadence, expects different artifacts, and applies a different evaluation lens. Table 2.1 summarizes how these expectations are managed throughout the project.

Stakeholder	Cadence	Alignment Approach	Proof Artifacts
Academic supervisor	Bi-weekly	Clear scope definition, documented architectural decisions, early report drafts	Control Center, Decision Register, report drafts
Professional supervisor	Weekly	Structured progress updates, visible deliverables, scope and deadline discipline	Execution log, KPI dashboard, demos
GHI technical and leadership stakeholders	Weekly	Incremental functional demos, focus on lifecycle use cases observed during audit	Scrumban board, demo recordings, execution log
Academic jury	Defense only	Structured report, honest discussion of scope and limits, defended validation results	Final report, defense slides, validation evidence

Table 2.1: Stakeholder alignment matrix showing the cadence, alignment approach, and proof artifacts for each project stakeholder

System Actors

The actors that interact with Itqan correspond to the roles observed at GHI during the audit and to the additional administrative roles that any production deployment requires. Six actors are distinguished, each with responsibilities that the platform must support.

The **Estimation manager** oversees the RFQ portfolio, performs the initial feasibility check at intake, classifies incoming RFQs, and approves bids before submission to the client. The **Estimator** performs the detailed cost work for each RFQ assigned to them, advances the RFQ through the cost estimation phase, and submits the completed workbook for internal review. The **Department user** represents the engineering, production, quality control, and procurement contributors who provide inputs to specific stages of the lifecycle and whose blockers must be visible to the rest of the organization. The **Executive** is a leadership-level actor (typically the General Manager of GHI or senior leadership at GHI / Albassam Group) who consumes pipeline-level views, win-loss analytics, and bottleneck reports without typically operating on individual RFQs. The **Administrator** manages user accounts, role assignments, workflow templates, and platform configuration. The **Copilot user** is a cross-cutting role rather than a separate person: any of the actors above can interact with the platform through the conversational layer to query the state of an RFQ, retrieve evidence-grounded answers about the lifecycle, or surface analytical observations.

The use case diagram in Section 2.6 presents how these actors interact with the system at a higher level of abstraction.

2.3 Functional Requirements

The functional requirements of Itqan describe the operational capabilities the platform must offer to the actors identified in Section 2.2. They are derived directly from the audit findings of Chapter 1: each pain point identified there shapes one or more requirements that the platform addresses. Requirements are organized by module to make their mapping to implementation

straightforward, and each requirement carries a unique identifier that is referenced in the traceability matrix of Section 2.8 and in the validation evidence of Chapter 5.

Manager Service Requirements

The manager service provides the operational backbone of the platform. Its requirements correspond to the lifecycle management responsibilities formerly handled through scattered Excel files, email threads, and verbal coordination. Table 2.2 lists the functional requirements assigned to this module.

ID	Requirement
FR-MGR-01	The system shall allow authorized users to register a new RFQ with its identifying metadata, source channel, and classification.
FR-MGR-02	The system shall maintain a structured representation of each RFQ's lifecycle stages, instantiated from a workflow template at creation time.
FR-MGR-03	The system shall enforce stage progression rules, including mandatory field completion and blocker resolution before stage advancement.
FR-MGR-04	The system shall support attachment of files, notes, and subtasks to any stage of an RFQ, with append-only semantics for notes.
FR-MGR-05	The system shall track manual and rule-based reminders associated with stages or subtasks, with assignee and due date.
FR-MGR-06	The system shall expose analytical endpoints that aggregate RFQs by status, classification, stage, and other operationally meaningful dimensions.
FR-MGR-07	The system shall preserve traceability of lifecycle changes through timestamps, status history on advancing stages, and non-destructive update semantics on critical records; comprehensive audit-log expansion is treated as a hardening concern beyond the current scope.
FR-MGR-08	The system shall support soft deletion of subtasks with automatic propagation to parent progress calculations.

Table 2.2: Functional requirements of the manager service

Copilot Service Requirements

The copilot service provides the conversational layer through which actors interact with the platform in natural language. Its requirements emphasize evidence-grounded answers and

explicit scope discipline rather than raw conversational fluency. Table 2.3 lists the functional requirements assigned to this module.

ID	Requirement
FR-COP-01	The system shall accept natural language queries from authorized users and produce responses grounded in platform evidence.
FR-COP-02	The system shall support two distinct entry contexts: a portfolio-level mode and an RFQ-bound mode tied to a specific RFQ.
FR-COP-03	The system shall classify each turn into a known interaction path before generating a response.
FR-COP-04	The system shall ground each factual claim in the evidence source authorized for the active path, primarily manager service state in the implemented operational paths, with intelligence service artifacts incorporated as those paths become available.
FR-COP-05	The system shall refuse to answer queries that fall outside its supported scope, returning a structured refusal with a reason rather than fabricating an answer.
FR-COP-06	The system shall route every retrieval through manager-mediated access checks, so that the copilot cannot retrieve or expose data outside the actor context available to the requesting user.
FR-COP-07	The system shall maintain bounded conversation history and working memory within a thread, with explicit scoping by RFQ or by portfolio context.
FR-COP-08	The system shall verify the assistant's draft response against retrieved evidence before delivering it to the user.

Table 2.3: Functional requirements of the copilot service

Intelligence Service Requirements

The intelligence service provides deterministic parsing and analytical capabilities over the documents that surround an RFQ, primarily the Material Requisition package and the estimation workbook. The requirements presented here are the *current-scope foundation requirements*: they cover the parsing capabilities and the briefing produced from them, both of which are implemented and validated in this work. The richer analytical and predictive capabilities sketched by the four-pillar vision in Section 1.6, including historical pattern matching, supplier intelligence, and cost-range estimation, are deliberately excluded from these requirements and remain future scope. Table 2.4 lists the foundation requirements assigned to this module.

ID	Requirement
FR-INT-01	The system shall parse Material Requisition packages structured according to the standard sixteen-section layout and produce a deterministic profile of their contents.
FR-INT-02	The system shall parse estimation workbooks structured according to the thirty-six-sheet template and extract identity, item, cost-breakdown, and schedule profiles.
FR-INT-03	The system shall produce an intelligence briefing that summarizes parsed content for downstream presentation to estimators and the conversational layer.
FR-INT-04	The system shall surface anomalies and cross-check failures detected during parsing as structured review flags rather than silent corrections.

Table 2.4: Functional requirements of the intelligence service

User Interface and Cross-Cutting Requirements

A small number of requirements apply to the user interface or cut across multiple services. They are listed together in Table 2.5 for completeness.

ID	Requirement
FR-UI-01	The user interface shall present a portfolio view of RFQs filtered and sorted by operationally meaningful attributes.
FR-UI-02	The user interface shall present a detail view of each RFQ with its full lifecycle, attached artifacts, and copilot entry point.
FR-UI-03	The user interface shall surface key performance indicators relevant to the executive role, drawn from the manager service’s analytical endpoints.
FR-X-01	The system shall be deployable as a set of containerized microservices, each independently startable and observable.
FR-X-02	The system shall expose health and readiness probes on every service to support orchestration and operational monitoring.

Table 2.5: User interface and cross-cutting functional requirements

2.4 Non-Functional Requirements

The non-functional requirements of Itqan describe the qualities the platform must exhibit independent of any specific feature. They reflect both standard concerns of any production-grade software system, such as reliability, security, and maintainability, and concerns specific to a system whose conversational layer relies on a large language model. The latter category includes explicit guarantees about explainability, grounding in platform evidence, and the absence of fabricated content. Each requirement carries a unique identifier referenced in the traceability matrix of Section 2.8.

For brevity, the requirements are presented in a single grouped table organized by category. Five categories are distinguished: *Reliability and Traceability* (REL), capturing the observability and predictability of state changes; *Security and Access Control* (SEC), capturing service-layer authorization independent of the calling client; *AI Integrity* (AI), capturing the integrity guarantees

that distinguish the copilot from a generic conversational agent; *Maintainability and Architecture* (MNT), capturing the architectural discipline required to evolve the platform’s services independently; and *Operability* (OPS), capturing the deployment, observability, and operational characteristics required to run the platform beyond the developer’s machine. Some non-functional requirements describe target behavior whose full validation is contingent on capabilities deferred to future scope, including the dedicated identity service noted in Section 1.6 and Section 2.5; this is acknowledged in the relevant rows.

ID	Category	Requirement
NFR-REL-01	Reliability	State-changing operations on lifecycle records shall preserve traceability through timestamps and non-destructive update semantics; the dedicated audit-log surface is a hardening concern beyond the current scope.
NFR-REL-02	Reliability	The system shall handle transient infrastructure failures gracefully by preserving already persisted state, returning structured errors, and allowing the failed operation to be retried safely.
NFR-REL-03	Reliability	Soft-deleted artifacts shall remain retrievable for review purposes; no destructive deletion shall occur on critical lifecycle records.
NFR-REL-04	Reliability	Errors shall be reported with a structured, correlatable identifier that permits end-to-end tracing across services.
NFR-SEC-01	Security	The system shall be designed so that role-based access control can be enforced at the service layer independently of the calling client. In the current scope, access behavior is exercised through actor-context headers and manager-mediated retrieval; full integration with a dedicated identity service is deferred.
NFR-SEC-02	Security	The copilot service shall not bypass the access controls of the manager service: every retrieval shall be mediated through the manager's authorized endpoints.
NFR-SEC-03	Security	Authentication credentials and external API keys shall not appear in logs, responses, or persisted records.
NFR-SEC-04	Security	The schema and queries shall be structured so that per-user scoping of retrievals is achievable at the data layer; full enforcement of cross-user data isolation is contingent on the dedicated identity service deferred to future scope.
ENET'COM SFAX		

2.5 Product Backlog

The product backlog presents the work scoped for the project from the perspective of the actors that benefit from it. Each backlog item is expressed as a user story following the conventional format *As a [actor], I want [capability], so that [value]*, and is assigned a priority that reflects its centrality to the project’s narrative and its readiness for development. The backlog is not a fixed plan: items have been added, refined, and reprioritized throughout the project as the audit findings translated into concrete capabilities and as the strategic pivot reshaped what the platform should deliver. The version presented in this section reflects the consolidated state at the project’s mid-point, after the first vertical slices had been delivered and validated.

Backlog items are grouped by module to align with the requirements organization in Sections 2.3 and 2.4. Priority is expressed on three levels: *High* for items central to the demonstration narrative and required for the defense, *Medium* for items that strengthen the platform without being defense-critical, and *Low* for items that are scoped but deferred to future iterations.

Manager Service Backlog

The manager service backlog covers the operational backbone of the platform. Its items are uniformly high-priority because the entire platform depends on the lifecycle representation this service provides.

ID	User Story	Priority
B-MGR-01	As an Estimation Manager, I want to register an incoming RFQ with its source, classification, and identifying metadata, so that it enters the lifecycle in a structured and trackable form.	High
B-MGR-02	As an Estimator, I want to advance an RFQ through its lifecycle stages with mandatory fields and blocker checks, so that the workflow enforces the discipline that was previously informal.	High
B-MGR-03	As a Department User, I want to attach notes, files, and subtasks to a stage of an RFQ, so that my contributions are visible in context to the rest of the team.	High
B-MGR-04	As an Estimator, I want to set reminders on stages or subtasks, so that follow-up actions are not lost in private notebooks or scattered email threads.	High
B-MGR-05	As an Executive, I want analytical endpoints that aggregate the RFQ portfolio by status, classification, and stage, so that I have visibility into the pipeline without depending on verbal updates.	High
B-MGR-06	As an Administrator, I want to define and version workflow templates, so that the lifecycle structure can evolve without breaking existing RFQs.	Medium

Table 2.7: Manager service backlog

Copilot Service Backlog

The copilot service backlog reflects the iterative development of the conversational layer. Early items established the pipeline and the first end-to-end path; later items broadened path coverage and refined the trust-boundary enforcement.

ID	User Story	Priority
B-COP-01	As a Copilot User, I want to ask natural-language questions about a specific RFQ and receive answers grounded in platform data, so that I can investigate the lifecycle without navigating multiple screens.	High
B-COP-02	As a Copilot User, I want the copilot to refuse questions outside its scope rather than fabricating an answer, so that I can trust the responses I receive.	High
B-COP-03	As a Copilot User, I want my conversation to maintain bounded context within the active thread, so that I can build on recent questions naturally without forcing the copilot to guess.	High
B-COP-04	As a Copilot User, I want the copilot to handle ambiguous or incomplete questions by asking for clarification rather than guessing, so that the answers remain reliable.	Medium
B-COP-05	As a Copilot User, I want a portfolio-level conversational entry point in addition to the RFQ-bound mode, so that I can explore the broader pipeline conversationally.	Medium
B-COP-06	As a Copilot User, I want the copilot to verify its draft response against retrieved evidence before delivering it, so that hallucinations are caught before they reach me.	Medium

Table 2.8: Copilot service backlog

Intelligence Service and Cross-Cutting Backlog

The intelligence service backlog within the present scope focuses on parsing capabilities; the predictive components remain future work. The cross-cutting items cover the platform-level concerns of containerization, observability, and the demonstration user interface.

ID	User Story	Priority
B-INT-01	As an Estimator, I want the platform to parse Material Requisition packages deterministically and surface their content, so that I can quickly orient on a new RFQ without manually reading every file.	High
B-INT-02	As an Estimator, I want the platform to parse the estimation workbook and extract structured cost and identity profiles, so that downstream services can reason over the workbook content.	High
B-INT-03	As an Estimator, I want the platform to flag anomalies and cross-check failures detected during parsing, so that review-worthy artifacts are not silently accepted.	Medium
B-UI-01	As any actor, I want a clean web interface that exposes the platform's portfolio view, RFQ detail view, and copilot entry point, so that the system is usable without API tools.	High
B-X-01	As an Administrator, I want each service deployable as a container with externalized configuration, so that the platform can be brought up reproducibly.	High
B-X-02	As an Administrator, I want each service to expose readiness and metrics endpoints, so that the platform's operational health is observable.	Medium
B-X-03	As an Administrator, I want a dedicated identity service to centralize authentication and role management, so that access control is not duplicated across services.	Low

Table 2.9: Intelligence service and cross-cutting backlog

The relative priority of items reflects the demonstration-readiness of the platform rather than the long-term importance of each capability. The single Low-priority item, the dedicated identity service (B-X-03), is acknowledged here because it appeared in the four-pillar vision but was deliberately deferred: its functionality is currently provided by a minimal authentication shim within the manager service, and its extraction into a dedicated service belongs to the future scope established in Section 1.6.

2.6 Use Cases and Activity Diagrams

This section presents the behavioral view of Itqan. The use case diagram in Figure 2.1 shows how the actors identified in Section 2.2 interact with the major capabilities of the platform. The activity diagrams that follow detail the two workflows whose internal logic best characterizes the project: the advancement of an RFQ through its lifecycle stages, which exercises the discipline of the manager service, and the conversational interaction with the copilot, which exercises the trust-bound pipeline that distinguishes this work from generic conversational systems.

System-Level Use Case Diagram

The use case diagram condenses the most consequential interactions between actors and the platform. It is not exhaustive: it focuses on the use cases that anchor the demonstration narrative and that carry traceable functional requirements. Use cases reachable through both the user interface and the conversational layer are shown as connected to both, reflecting the cross-cutting nature of the copilot user role established in Section 2.2.

[Figure 2.1 placeholder: system-level use case diagram showing the actors and the major use cases of the platform]

Figure 2.1: System-level use case diagram of the Itqan platform

The major use cases visible in the diagram are: *Register RFQ*, *Advance RFQ Stage*, *Manage Stage Artifacts* (notes, files, subtasks), *Configure Reminders*, *Consult Pipeline Analytics*, *Query Copilot in Portfolio Mode*, *Query Copilot in RFQ-Bound Mode*, and *Manage Workflow Templates*. Each use case is realized by a combination of services described in Chapter 3.

Activity Diagram: Advance RFQ Stage

The advancement of an RFQ from one stage to the next is the most consequential workflow in the manager service. It is the workflow where the platform's lifecycle discipline becomes concrete: a stage cannot be advanced if its mandatory fields are incomplete, if it has unresolved blockers, or if its computed next stage is not reachable from the current state. When advancement succeeds, the change propagates: the parent RFQ's progress is recomputed, the next stage is activated, and a status history entry is written. Figure 2.2 shows the full flow.

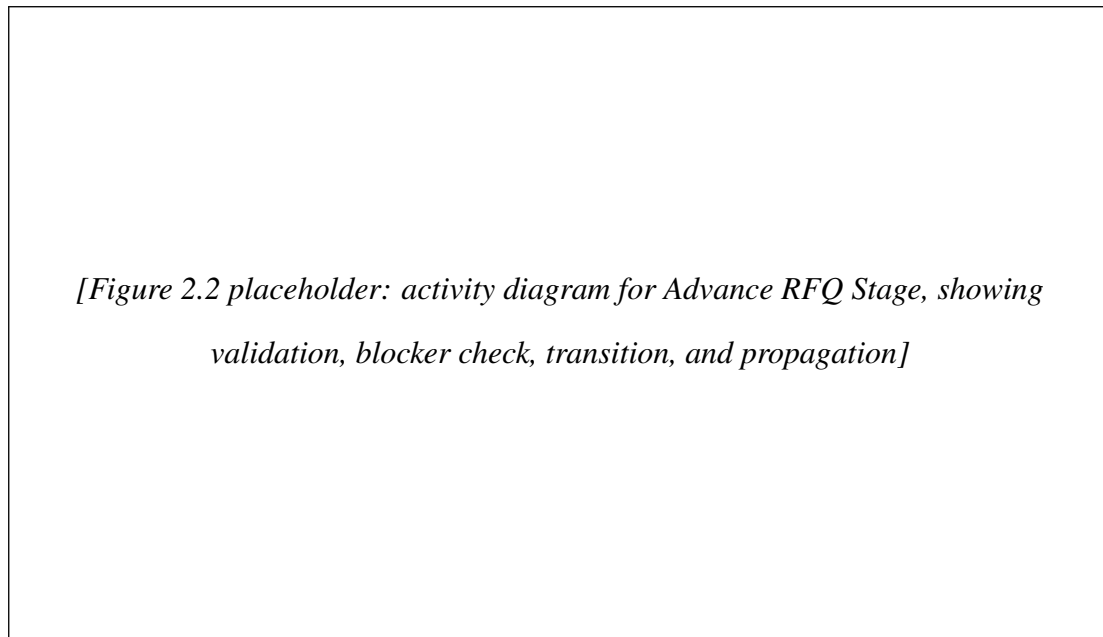


Figure 2.2: Activity diagram for the *Advance RFQ Stage* use case

The diagram exposes the three decision points that the manager service enforces deterministically: the mandatory-field gate, the blocker gate, and the transition-validity gate. Each gate corresponds to a private method in the controller layer of the manager service, presented in detail in Chapter 4.

Activity Diagram: Ask Copilot About an RFQ

The conversational interaction with the copilot is structurally different from the lifecycle workflows of the manager service. A single user turn passes through a sequence of stages, several of which are deterministic policy checks rather than language-model invocations. Figure 2.3 presents the activity flow at a level of abstraction appropriate for this chapter; the detailed component architecture and the trust boundaries enforced at each stage are developed in Chapter 3.

[Figure 2.3 placeholder: activity diagram for Ask Copilot About an RFQ, showing classification, evidence retrieval, composition, and verification stages, with deterministic stages clearly distinguished from language-model invocations]

Figure 2.3: Activity diagram for the Ask Copilot About an RFQ use case

The activity diagram makes visible a property of the copilot that distinguishes it from generic conversational systems: the language model is invoked at specific, bounded stages of the flow (classification of the user turn, composition of the draft response, and verification of the draft) and not as the sole agent driving the interaction. The deterministic stages that surround these invocations are where the trust-boundary architecture operates.

2.7 Methodology and Project Management

The methodology that structured this project responds to a specific constraint: a single developer working over several months across the platform’s services, with regular but asynchronous interaction with two supervisors and a remote industrial client. This constraint excludes both the heavy ceremony of pure Scrum, which depends on a co-located team, and the absence of structure that often accompanies solo work. The practices presented in this section were chosen and adapted to fit this constraint while preserving the engineering rigor that the project requires. They are presented in three parts: the iteration framework that organizes the work, the decision register that captures architectural choices as they are made, and the risk practice that surfaces threats before they become incidents.

Scrumban as the Iteration Framework

The iteration framework adopted is Scrumban, a hybrid that retains the cadence and demo discipline of Scrum while substituting Kanban-style work-in-progress limits for the rigidity of fixed-length sprints. The hybrid form is well suited to a solo PFE for two complementary reasons.

The first reason is that the social mechanisms of Scrum are not available. There is no team for daily standups, no peer to review pull requests in real time, and no scrum master to remove impediments. The discipline that these mechanisms enforce in a team setting must come from elsewhere when the work is solo. Kanban-style work-in-progress limits provide that discipline structurally: the developer cannot start a new task while too many tasks are in flight, which prevents the most common failure mode of solo software work, namely the simultaneous spreading of attention across too many parallel threads.

The second reason is that the demo cadence of Scrum is precisely what stakeholder alignment requires. The professional supervisor and the GHI technical stakeholders both expect visible, demonstrable progress on a regular schedule. Pure Kanban does not impose such a cadence; pure Scrum imposes it but at the cost of fixed-duration sprints that can either truncate or pad work artificially. Scrumban's hybrid structure preserves the demo discipline while allowing iteration boundaries to align with natural completion points of the work.

The Scrumban board used throughout the project follows a six-state workflow: *Backlog (Frozen Scope)*, *Ready*, *In Progress*, *Review and Demo*, *Done*, and *Documented*. The presence of a *Documented* terminal state, distinct from *Done*, deserves explicit mention. Most Scrumban implementations treat documentation as part of the definition of done or, more commonly, as an afterthought to be addressed later. In this project, documentation is treated as a separate column with its own work-in-progress limit. A task is not removed from the active board until its documentation, whether code-level (docstrings, READMEs), architectural (decision records, design notes), or report-level (chapter draft updates), has been written. This discipline is what permits the chapter content of this report to track development continuously rather than being

assembled at the end. Figure 2.4 presents the consolidated sprint timeline of the project from the start of development in January.

[Figure 2.4 placeholder: consolidated sprint timeline showing the six-state Scrumban workflow across the project's iterations from January onward]

Figure 2.4: Consolidated sprint timeline of the project, showing iteration boundaries and the distribution of work across the platform's services

Architectural Decision Records

A solo PFE produces architectural decisions at high frequency. The decision to adopt the BACAB layered pattern, the decision to keep reminders inside the manager service rather than splitting them out, the decision to use a single ExecutionPlanFactory in the copilot, the decision to defer the dedicated identity service to future work: each of these is a real fork in the project's evolution, with alternatives that were considered and consequences that became apparent later. Without explicit capture, these decisions are quickly forgotten and the rationale behind them is lost.

The project therefore maintains an Architectural Decision Record (ADR) register. Each entry follows a consistent structure: the *context* that triggered the decision, the *options considered* with their respective pros and cons, the *decision* itself stated in a single sentence, the *justification* that connects the decision to the specific constraints of this project, the *consequences* (both positive and negative) accepted by adopting the chosen option, and the *validation* mechanism through which the decision is or will be confirmed. The register is referenced throughout Chapter 3, where major architectural decisions are introduced with their ADR identifier and a pointer to the register entry.

The discipline of recording decisions formally rather than informally serves three purposes that justify the overhead in a solo context. It forces the consideration of at least one alternative before committing, which often surfaces a better option than the first one considered. It produces an artifact that supervisors and the jury can examine to evaluate the engineering judgment exercised, independent of whether they agree with each individual decision. And it leaves a trail that supports honest discussion of limitations and future work in Chapter 5.

Risk Management

A risk register accompanies the decision register. Where the decision register documents what was chosen, the risk register documents what could go wrong with what was chosen and what countermeasures are in place. Each risk entry follows a four-part structure: the *description* of what could occur, the *impact* if it does, the *mitigation* actions taken or planned to reduce its probability or severity, and *review notes* updated as the risk evolves.

In the solo PFE context, the risk register is less about insurance against organizational failures than about disciplined acknowledgment of known fragilities. Risks tracked in the register include scope creep beyond the implemented baseline, dependence on third-party API stability for the language model layer of the copilot, the limited scale of the validation dataset, and the academic timeline pressure relative to the platform's full vision. These risks are revisited at each iteration boundary; significant changes are discussed with the academic and professional supervisors during the regular alignment cadences described in Section 2.2. Specific risks materially affecting the project's deliverables are presented again in Chapter 5 as part of the limitations discussion.

The Control Center

The Scrumban board, the decision register, the risk register, the stakeholder alignment matrix, and the sprint execution log are maintained together in a project Control Center that serves as the single operational hub for the project. The Control Center is not part of the deliverable platform; it is the project management surface through which the work is planned,

tracked, and documented. Its existence as a unified surface, rather than as fragmented spreadsheets and chat threads, is itself a small example of the discipline this chapter argues the project requires.

2.8 Traceability Matrix

The traceability matrix presented in Table 2.10 links each functional requirement of Section 2.3 to the module that implements it and to the area of Chapter 5 where its validation evidence will be presented. The Chapter 5 references are provisional at this stage of the report and will be synchronized with the final section numbering once Chapter 5 is finalized.

The matrix serves three purposes. It provides the reader with a compact reference for navigating between requirements, implementation, and validation throughout the rest of the report. It exposes any requirement that lacks a clear implementation locus or a clear validation path, and therefore functions as a structural quality check on the project itself. And it supports the discussion of business impact in Chapter 5, where Itqan's capabilities are mapped back to the audit pain points of Chapter 1 by composing this matrix with the pain-point table in Section 1.4.

Non-functional requirements are not enumerated row by row in the matrix. Their validation is qualitative rather than scenario-driven, and is presented as part of the trust-boundary verification, the operability checks, and the architectural decisions discussed in Chapter 5. Specific NFRs of particular consequence (notably the AI integrity requirements) are revisited explicitly in that chapter.

Requirement	Capability	Module	Validation
FR-MGR-01	RFQ registration with metadata and classification	Manager	Ch. 5 (Manager)
FR-MGR-02	Workflow-driven stage instantiation	Manager	Ch. 5 (Workflow)
FR-MGR-03	Stage progression with mandatory fields and blocker checks	Manager	Ch. 5 (Workflow)
FR-MGR-04	Notes, files, subtasks attached to stages	Manager	Ch. 5 (Manager)
FR-MGR-05	Manual and rule-based reminders on stages and subtasks	Manager	Ch. 5 (Manager)
FR-MGR-06	Analytical aggregation endpoints	Manager	Ch. 5 (Manager)
FR-MGR-07	Lifecycle traceability through timestamps and history	Manager	Ch. 5 (Workflow)
FR-MGR-08	Soft-delete semantics with parent progress propagation	Manager	Ch. 5 (Workflow)
FR-COP-01	Natural-language queries with evidence-grounded responses	Copilot	Ch. 5 (Copilot Scenarios)
FR-COP-02	Portfolio and RFQ-bound conversation modes	Copilot	Ch. 5 (Copilot Scenarios)
FR-COP-03	Path classification of each turn	Copilot	Ch. 5 (Copilot Scenarios)
FR-COP-04	Path-authorized evidence grounding	Copilot	Ch. 5 (Trust Boundary)
FR-COP-05	Structured refusal of out-of-scope queries	Copilot	Ch. 5 (Copilot Scenarios)
FR-COP-06	Per-user access control on every retrieval	Copilot	Ch. 5 (Trust Boundary)

Two observations are worth surfacing from the matrix. The copilot requirements concentrate the largest fraction of validation activity in Chapter 5, reflecting the centrality of the conversational layer to the project's innovation and the corresponding need for rigorous evidence that its trust-boundary properties hold in practice. The intelligence service requirements all map to a single validation area because the present scope of that service is the parsing foundation, and the foundation is validated as a unit against the golden reference artifacts established during the audit.

2.9 CONCLUSION

This chapter has formalized the engineering structure that disciplined the project. It identified the four stakeholders whose alignment shapes the project's success criteria and the six actors that interact with the platform in operation. It catalogued twenty-five functional requirements organized by module, nineteen non-functional requirements covering reliability, security, AI integrity, maintainability, and operability, and a product backlog whose user stories reflect the actors' perspectives and the project's priorities. It presented the use case decomposition and the activity diagrams that capture the most consequential workflows of the platform. It explained the methodology adopted, including Scrumban as the iteration framework with its distinctive *Documented* terminal state, the architectural decision register that captures choices as they are made, and the risk register that surfaces fragilities before they materialize. It closed with a traceability matrix that links each functional requirement to its implementing module and to the validation evidence presented in Chapter 5. The next chapter develops the conceptual and technical architecture that delivers on these requirements, with particular emphasis on the trust-bound copilot architecture as the principal innovation contribution of the project.

Conceptual and Technical Architecture

Contents

3.1	INTRODUCTION	46
3.2	Itqan Global Architecture and Four-Pillar Vision	46
3.3	Microservices Architecture	50
3.4	RFQ Manager Service: Lifecycle Orchestration Backbone . .	53
3.5	RFQ Copilot Service: Trust-Bound Conversational Architecture	58
3.6	Evidence Boundary and Source-of-Truth Strategy	67
3.7	Data Model and API Contracts	71
3.8	Platform Integration: UI and Intelligence Layers	74
3.9	Architectural Decisions and Trade-offs	76
3.10	CONCLUSION	78

3.1 INTRODUCTION

This chapter presents the conceptual and technical architecture of Itqan. It moves from the platform level to the service level and back to the integration level, and it spends its longest section on the architecture of the conversational layer that is the principal innovation contribution of this work. The chapter does not describe code or implementation details; those are the subject of Chapter 4. It describes components, responsibilities, boundaries, the data and evidence each component owns, and the trade-offs accepted in arriving at the present design.

The chapter opens with the platform-level view and the four-pillar vision, distinguishing what is implemented in this work from what is future scope. It then presents the microservices architecture and the layered pattern that disciplines each service internally. Two services are then developed in depth: the manager service, which is the operational backbone of the platform, and the copilot service, whose trust-boundary architecture is the central innovation of the project. The chapter continues with the evidence boundary that governs how the copilot reasons over platform data, the data model and API contracts that anchor inter-service communication, and the platform integration layer that composes the services into a coherent whole. It closes with a summary of the major architectural decisions made during the project and a transition to the implementation chapter.

3.2 Itqan Global Architecture and Four-Pillar Vision

Itqan is a platform built around four interdependent capabilities, organized internally as a layered architecture and externally as a set of cooperating microservices. This section presents the platform at the highest level of abstraction: the four pillars that define its long-term scope, the layered architecture that organizes its internal logic, and the explicit scope status of each pillar in the present work. The detailed service-level architecture is the subject of Sections 3.3 through 3.8.

The Layered Architecture

At the platform level, Itqan organizes its components into four layers, each with a distinct responsibility and a distinct data ownership posture. The **presentation layer** consists of the user interface service and the conversational layer, which together expose the platform to its actors. The **orchestration layer** consists of the manager service, which owns the lifecycle representation of every RFQ and the operational truth that derives from it. The **intelligence layer** consists of the intelligence service, which produces deterministic analytical artifacts from the documents that surround an RFQ but does not own any lifecycle state. The **data layer** consists of the persistence stores managed by each service, kept private to that service to preserve the integrity of the service boundaries. Figure 3.1 presents the layered architecture and the cross-cutting position of the conversational layer that traverses presentation, orchestration, and intelligence.

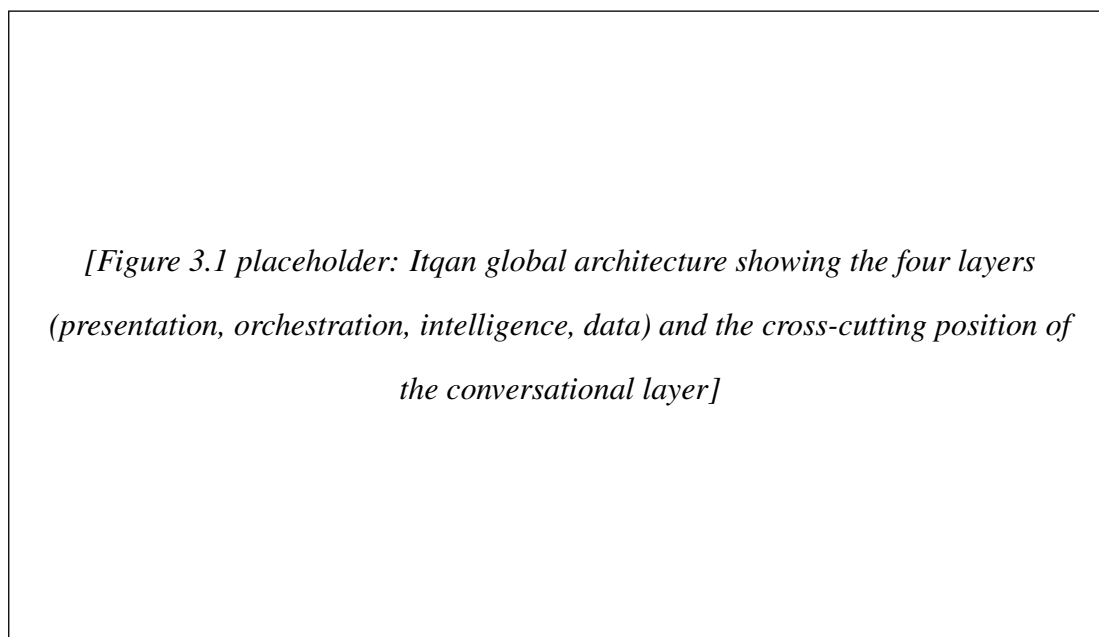


Figure 3.1: Global architecture of the Itqan platform: four layers and the cross-cutting conversational layer

The discipline of the layered architecture is not formal for its own sake. It exists to enforce a property that the audit findings of Chapter 1 made central: that the platform must produce a structured, observable, and traceable representation of the RFQ lifecycle, owned by a single component, on which all other components depend. The orchestration layer holds that representation.

The presentation and intelligence layers consume it; they do not duplicate it or compete for ownership of it.

The Four Pillars as an Architectural Map

The four pillars introduced in Chapter 1, Section 1.6, are revisited here as an architectural map rather than as a vision statement. Each pillar corresponds to a coherent set of capabilities, a primary owning service, and a set of pain points it addresses. The four pillars are not orthogonal: they share data, they reference each other, and the conversational layer cuts across all four.

The **Workflow and Tracking** pillar is owned by the manager service. It is the operational backbone of the platform: it maintains the lifecycle representation of every RFQ, enforces stage progression rules, and provides the analytical aggregations on which downstream visibility depends. Every other pillar, directly or indirectly, depends on the lifecycle representation that this pillar owns.

The **Communication Automation** pillar is co-owned by the manager service (for reminders tied to lifecycle artifacts) and a future communication service (for outbound channels and unified inbound monitoring). The decision to keep reminders inside the manager service rather than splitting them into a separate service from the outset is itself an architectural decision, discussed in Section 3.9.

The **Executive Intelligence and BI** pillar is co-owned by the manager service (for analytical aggregations over lifecycle data) and the user interface service (for the dashboards that consume those aggregations). The boundary between data production and data presentation is preserved: aggregations live in the manager, dashboards live in the UI.

The **Historical Learning and Prediction** pillar is owned by the intelligence service. It is the longest-horizon pillar: its short-term scope is the deterministic parsing of the artifacts that surround an RFQ, and its long-term scope is the predictive layer that would learn from accumulated history. The current work delivers the parsing foundation; the predictive layer is future scope.

Scope Status of the Four Pillars

A precise statement of what is implemented, what is partial, and what is future is essential to the honest framing of this chapter. Table 3.1 presents the scope status of each pillar as it stands at the conclusion of the present work.

Pillar	Status	Notes on Coverage
Workflow and Tracking	Implemented	Operational backbone of the platform, delivered through the manager service across all primary lifecycle resources.
Communication Automation	Partial	Reminder infrastructure delivered inside the manager service; unified inbound monitoring and assisted outbound drafting are future scope.
Executive Intelligence and BI	Partial	Analytical endpoints delivered in the manager service; demonstration dashboards consumed by the UI; full BI surface, including profitability and competitive analysis, is future scope.
Historical Learning and Prediction	Foundation only	Deterministic parsing of MR packages and estimation workbooks delivered in the intelligence service; pattern matching, supplier intelligence, and predictive components are future scope.

Table 3.1: Scope status of the four pillars at the conclusion of the present work

The cross-cutting conversational layer, the RFQ Copilot, is implemented within the present scope and is the principal innovation contribution of the project. Its detailed architecture is the subject of Section 3.5. The copilot is not a fifth pillar: it does not own data of its own, but rather navigates and explains the data that the four pillars produce. This positioning is fundamental to

the trust-boundary architecture and is the reason the copilot is treated as a cross-cutting layer rather than a pillar in its own right.

3.3 Microservices Architecture

At the service level, Itqan is composed of a set of cooperating microservices, each owning a clearly defined domain and exposing its capabilities through versioned API contracts. This section presents the service inventory, the rationale for the chosen boundaries, the layered pattern that disciplines each service internally, and the conventions that govern inter-service communication. The detailed architecture of the two services that carry the technical weight of this work, the manager and the copilot, is developed in the sections that follow.

Service Inventory and Boundaries

The platform is designed around six services, distinguished by maturity rather than by binary implementation status. Four services belong to the present project surface at different maturity levels. The two services that carry the technical weight of this work are `rfq_manager_ms`, the lifecycle orchestration backbone, and `rfq_copilot_ms`, the trust-bound conversational layer; both are substantially implemented and exercised end-to-end through the operational paths described in Chapter 4 and the validation activities in Chapter 5. The other two services are `rfq_intelligence_ms`, the deterministic parsing and analytical foundation, which is implemented at the foundation level for the parsing scope established in Chapter 2 and remains read-heavy and analytical in posture, and `rfq_ui_ms`, the user interface that exposes the platform to its actors at the demonstration level required to support the defense narrative. The remaining two services, `rfq_iam_ms`, a dedicated identity and access management service, and `rfq_communication_ms`, a dedicated communication and outbound messaging service, are positioned as future scope. Their absence from the present scope was a deliberate decision, discussed in Section 3.9, and their functionality is currently provided either by minimal in-service shims (authentication inside the manager) or by deferred capability (full unified inbox monitoring).

The boundary between services is drawn along the axis of data ownership rather than the axis of feature decomposition. The manager service owns lifecycle state. The intelligence service owns analytical artifacts derived from documents. The copilot service owns conversation state, and only conversation state: every fact about RFQs that the copilot returns is retrieved from the manager or the intelligence service at the moment of the response, never persisted by the copilot itself. The user interface service owns presentation logic but no domain data. This ownership discipline is what makes the trust-boundary architecture of the copilot possible: because the copilot owns no operational truth, it cannot drift from operational truth.

Figure 3.2 presents the service map and the principal inter-service communication paths.

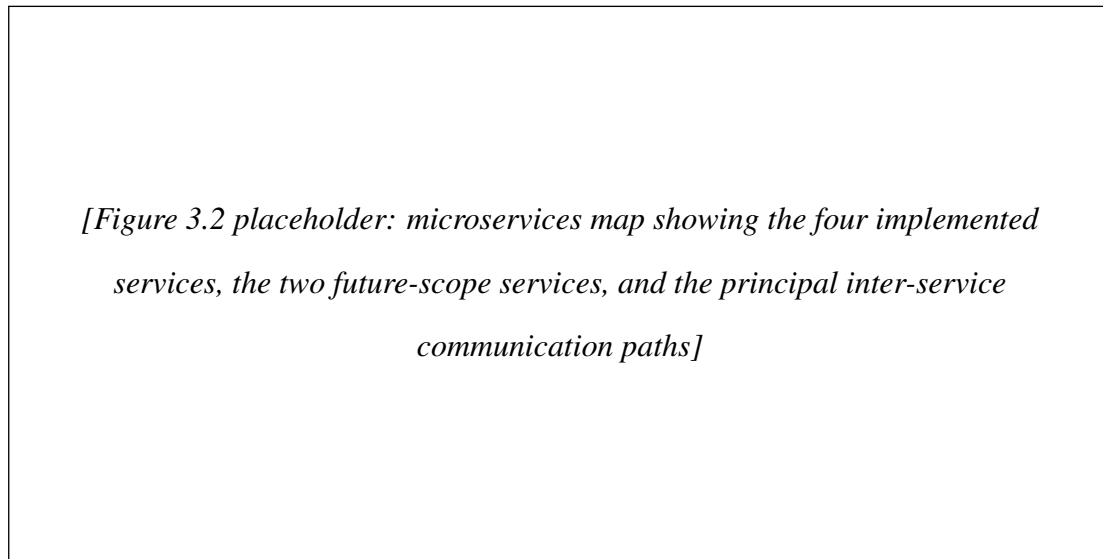


Figure 3.2: Microservices architecture of the Itqan platform, showing service boundaries and inter-service communication

The BACAB Layered Pattern

Inside each service, the code is organized according to a consistent layered pattern, derived from the architectural conventions used by BACAB Consulting across its engagements and adapted to the constraints of the present project. The pattern names five active layers and three support layers.

The active layers form the request path. The **routes** layer is the entry point: it receives HTTP requests, parses them into validated input, and delegates to the controllers. The **controllers**

layer holds the business logic: it orchestrates the steps required to satisfy a request, validates business rules, and decides what data must be read or written. The **datasources** layer is the persistence boundary: it executes database operations on behalf of the controllers and returns domain objects. The **translators** layer converts between internal domain representations and the shapes that cross service boundaries (API request and response schemas). The **connectors** layer encapsulates calls to external services, including the language model API consumed by the copilot.

The support layers serve the active layers without being part of the request path. The **models** layer defines the domain entities and their relational structure. The **config** layer centralizes service configuration drawn from the environment. The **utils** layer provides helpers that do not belong to any specific layer.

The discipline this pattern enforces is the same in every service: routes do not contain business logic, controllers do not access the database directly, and datasources do not make decisions. This separation makes each service easy to reason about in isolation, easy to test, and easy to extend without violating its own boundaries. The implementation of this pattern in each of the active services is the subject of Chapter 4.

Inter-Service Communication

Inter-service communication in Itqan follows three conventions. The first convention is that inter-service communication is primarily synchronous over HTTP using REST. Asynchronous messaging through a message bus is deliberately not introduced as a general pattern at the present scale: the operational complexity it would add is not justified by the volume of inter-service traffic in a system whose dominant access pattern is conversational and request-driven. Event-driven communication is retained as an architectural evolution point for specific flows where it would be more appropriate, notably the workbook upload flow in which the manager service notifies the intelligence service that an estimation workbook is available for parsing; in the present implementation this flow is realized through a synchronous trigger but is structurally amenable to event-driven evolution as the platform matures.

The second convention is that all inter-service calls cross documented API contracts. Each service publishes an OpenAPI specification of its public surface, and consuming services rely only on that specification rather than on internal data structures. This decoupling is what permits each service to evolve internally without breaking its consumers.

The third convention concerns access control. The copilot service does not bypass the access controls of the manager service: every retrieval of lifecycle data initiated by the copilot is mediated through the manager's authorized endpoints, with the requesting user's actor context propagated explicitly. This convention, which Section 3.5 will revisit as the access boundary of the trust-bound architecture, is what keeps the copilot from becoming a privilege escalation vector.

3.4 RFQ Manager Service: Lifecycle Orchestration Backbone

The manager service is the operational backbone of the platform. It is the service on which every other service depends, and it is the service whose architecture is most directly answerable to the audit findings of Chapter 1. This section presents the manager service at the architectural level: its mission, its domain model, the workflow and stage progression mechanics that give it its discipline, and the decisions about what belongs inside it and what belongs outside.

Service Mission

The mission of the manager service is to maintain a structured, observable representation of every RFQ across its full lifecycle, and to enforce the rules that govern that lifecycle. This mission is more constrained than the phrase "RFQ management" might suggest. The manager service is not the place where RFQs are estimated; the estimation workbook remains in the hands of senior estimators, as established in Chapter 1. The manager service is also not the place where intelligence is produced; that is the role of the intelligence service. The manager service is the orchestration layer in the strict sense: it owns the state of the lifecycle, the rules that govern its progression, and the artifacts attached to it.

This narrowing of mission is itself an architectural choice. A more permissive scope, in which the manager service also held analytical, predictive, or conversational responsibilities, would have produced a service that was harder to reason about, harder to test, and harder to evolve. The deliberate constraint of mission to lifecycle orchestration is what makes the rest of the platform architecturally clean.

Domain Model

The domain model of the manager service is organized around seven primary entities, each corresponding to a clear concept in the audit findings. The **RFQ** is the root entity: it carries the identifying metadata of the quotation request, its classification, its source channel, and its current lifecycle status. The **Workflow** is a versioned template that defines the sequence of stages an RFQ passes through; it is shared across many RFQs but instantiated separately for each. The **RFQ_Stage** is an instance of a workflow stage tied to a specific RFQ, carrying the stage's progression state, its mandatory fields, its computed transition rules, and its operational metadata. The **Subtask** is a finer-grained unit of work attached to a stage, supporting soft-delete semantics and parent progress propagation. The **Note** is an append-only textual artifact attached to a stage, capturing decisions, observations, or context. The **File** is a persisted document attached to a stage, including (importantly) the estimation workbook itself once it is uploaded for tracking. The **Reminder** is a time-anchored notification associated with a stage or subtask, supporting both manual and rule-based creation modes. Figure 3.3 presents the entity-relationship diagram of the manager service.

[Figure 3.3 placeholder: entity-relationship diagram of the manager service, showing the seven primary entities and their cardinalities]

Figure 3.3: Entity-relationship diagram of the manager service domain model

Workflow and Stage Progression

The mechanism that gives the manager service its discipline is the stage progression logic, encoded in the controller layer and exercised through the API surface. The progression of an RFQ through its stages is not a free transition: it is a guarded one. Three deterministic gates must be passed before a stage can be advanced.

The first gate is the *mandatory-field gate*. Each stage carries a list of fields that must be populated before the stage can be marked complete. The mandatory-field list is part of the workflow template and can vary by stage. If a required field is missing, the gate refuses the advance with a structured error identifying the missing fields.

The second gate is the *blocker gate*. A stage may carry blocker state raised by department users, for example a feasibility blocker raised by Engineering. A stage cannot be advanced while its blocker state is unresolved. Blocker information is represented at the stage level through structured blocker fields, including a status indicator and a reason code where available. This representation allows the manager service to enforce the blocker gate without introducing a separate blocker resource in the current scope; promotion of blockers into a dedicated entity

is recognized as an evolution direction if the platform's communication and notification needs grow beyond what the stage-level representation supports.

The third gate is the *transition-validity gate*. The next stage of an RFQ is computed from the workflow template, not chosen arbitrarily by the user. If the requested target stage is not the computed next stage, the gate refuses the advance. This prevents stages from being skipped or revisited out of order.

When all three gates pass, the advance is committed: the current stage is marked complete with a timestamp, the next stage is activated, the RFQ's overall progress is recomputed, and a status history entry is written. This sequence is atomic from the caller's perspective. The progression logic is the mechanism through which the platform converts a sequence of expert acts into a coordinated process, addressing pain points PP-01, PP-05, and PP-08 from Chapter 1 directly. The implementation of the three gates in the controller layer of the manager service is presented in Chapter 4.

The Reminder Architectural Decision

A specific architectural decision deserves explicit treatment: the decision to keep reminders inside the manager service rather than to extract them into a separate communication or notification service.

The argument for extraction would be straightforward. Reminders are notifications. Notifications are communication. Communication is the responsibility of a communication service, which would also handle outbound messaging, channel monitoring, and assisted drafting in the long-term scope of Pillar 2. Extraction would align reminders with the eventual home of their feature family.

The argument against extraction, which prevailed, is grounded in the data ownership principle established in Section 3.3. Reminders in their current form are tightly coupled to lifecycle artifacts: they are attached to specific stages or subtasks, they fire based on stage progression rules and due dates, and they participate in the stage-completion lifecycle. Extracting them into a separate service would either require the communication service to cache lifecycle state

(violating ownership boundaries) or require every reminder operation to make a synchronous round trip to the manager service for context (creating a coupling that defeats the purpose of extraction).

The decision to keep reminders inside the manager service for the present scope is therefore a defense of the data ownership boundary. It is acknowledged as a decision rather than as a default: when the communication pillar matures and outbound channels become first-class concerns, the boundary may be revisited and reminders may migrate. For the present work, they belong where the data they describe lives. This decision is recorded in the architectural decision register summarized in Section 3.9.

API Contract Overview

The manager service exposes its capabilities through a versioned REST API organized around seven resource families: RFQ, Workflow, RFQ_Stage, Subtask, File, Note, and Reminder. The initial V1 contract defined twenty-eight endpoints; the working implementation evolved to a broader operational surface of approximately thirty-one endpoints after hardening and after extensions to the reminder and statistics families that emerged from demonstration feedback. The contract follows REST conventions: resources are addressed by stable identifiers, state-changing operations use the appropriate HTTP verbs, and responses carry sufficient context for downstream consumers to reason about the state of the lifecycle without secondary calls.

A subset of endpoints carries particular significance. The *advance stage* endpoint exercises the three-gate progression logic described above. The *add note* endpoint is append-only by contract: no update or delete is offered, preserving the integrity of the communication trail attached to a stage. The *upload file* endpoint distinguishes file types in its contract, including a designated type for the estimation workbook that is intended to trigger downstream intelligence parsing in the integrated platform flow. The full API surface, including request and response schemas and error semantics, is the subject of the implementation chapter.

3.5 RFQ Copilot Service: Trust-Bound Conversational Architecture

The RFQ Copilot is the principal innovation contribution of this work. It is the conversational layer through which authorized users interact with the platform in natural language, with strict guarantees about the evidence on which its answers are grounded and the boundaries of what it is permitted to do. This section presents the design philosophy of the copilot, positions it against the alternative conversational architectures that exist in the field, and develops the trust-boundary architecture that organizes its internal logic. The implementation of this architecture in code is the subject of Chapter 4; this section operates at the conceptual and architectural level.

Design Philosophy

The copilot was designed under a single guiding principle, which is also the design principle that determines every subsequent architectural choice in this section.

The copilot should feel like ChatGPT in fluidity, but must not behave like generic ChatGPT in authority. Its authority comes from platform evidence, not from the language model.

This principle separates two qualities that are typically conflated in conversational systems. *Fluidity* is what makes a system pleasant to interact with: natural language understanding, contextually appropriate replies, the ability to follow a thread across turns. *Authority* is what makes a system safe to act on: the assurance that its claims are grounded in real data, that its scope is bounded, and that its behavior is predictable. A generic conversational system optimizes for fluidity and produces authority as a fluent-sounding side effect, which is precisely what the audit findings of Chapter 1 warned against in an industrial estimation context.

The copilot is therefore designed so that fluidity is provided by the language model while authority is owned by deterministic platform components. The language model can interpret a question, compose a response, and verify a draft against retrieved evidence. It cannot select tools, decide which data the user is allowed to see, choose what evidence is relevant, decide when to give up on a question, or commit to a response without verification. Each of these

decisions is the responsibility of a specific code component or configuration artifact whose behavior is deterministic.

Positioning Against Alternative Architectures

The trust-bound architecture is best understood by contrast with the three principal alternatives a conversational layer of this kind might adopt.

The first alternative is the **naïve retrieval-augmented generation (RAG) architecture**, in which user queries trigger a vector retrieval over a corpus of documents, and the retrieved passages are appended to the prompt for a language model to compose a response. Naive RAG is well suited to question-answering over static documents but struggles in a context where the answer depends on live operational state: lifecycle progression, access permissions, or recent state changes. It also offers no structural defense against the language model going beyond what was retrieved, because the model is the sole agent producing the answer.

The second alternative is the **generic LLM agent architecture**, in which a language model is given a set of tools and the autonomy to decide which tools to invoke, in what order, and with what arguments. This pattern is increasingly popular and visibly effective in low-stakes contexts. In a context where every tool call may expose commercially sensitive data, every retrieved value may end up in a contractual quotation, and every refusal must be defensible after the fact, the autonomy of an LLM agent is a liability rather than a feature. The decisions that determine safety (which tool may run, which user may see what, when to refuse) are the decisions that should not be made by the language model.

The third alternative is the **uncontrolled tool-calling chatbot**, a hybrid in which a language model classifies user intent and then triggers retrieval through a fixed set of endpoints, but without disciplined separation between classification, policy enforcement, and execution. This pattern is closer to safe but still permits the language model to influence policy decisions through its choice of classification, with no deterministic gate between intent and action.

The trust-bound copilot adopts none of these patterns. It accepts that language models are excellent at the linguistic tasks (interpretation, composition, verification) and unreliable at

the policy tasks (selection, authorization, escalation). It therefore allocates each decision to the component best suited to make it, separates classification from execution through a single deterministic chokepoint, and verifies the resulting answer through a sequence of deterministic guardrails followed by a final language-model judge.

The Eight Trust Boundaries

The architecture rests on eight decisions, each of which is consciously assigned to a specific owner. The trust boundaries are not implementation details; they are the architectural commitments that distinguish the copilot from the alternatives sketched above. Table 3.2 presents the eight boundaries and their owners.

#	Decision	Owner
1	What path does this turn belong to?	FastIntake (deterministic) or LLM Planner (proposes), validated by PlannerValidator and decided by ExecutionPlanFactory.
2	Who may construct an executable plan for a turn?	ExecutionPlanFactory only. CI-enforced single-construction discipline.
3	Which tools are permitted, and which actually run?	Path Registry (mapping) and the deterministic Tool Executor (invocation).
4	May this user read this RFQ?	The manager service API: a 404 or 403 response is the answer, never overridden by the copilot.
5	Which fields are allowed to enter the language model prompt?	Per-path field whitelist defined in the Path Registry, enforced by the ExecutionPlanFactory.
6	Which target does each field of evidence belong to?	Per-target labelling at the moment of evidence packet construction.
7	Did the response hallucinate, drift in scope, or leak across targets?	Deterministic Guardrails (multiple, sequential), followed by an LLM Judge as the final line of defense.
8	What happens when any stage fails?	The Escalation Gate, a single deterministic intercept point that re-enters the ExecutionPlanFactory with a structured reason_code.

Table 3.2: The eight trust boundaries of the copilot architecture and their owners

The mantra that summarizes these eight boundaries, used throughout the project as a quick test of architectural correctness, can be stated in one sentence: *two classification sources, one plan factory, one escalation gate; the language model produces language, code produces truth, policy enforces boundaries, the Judge verifies, templates render the safe answer when nothing else worked.* If a proposed change to the system would violate any clause of this sentence, it is the change that is wrong, not the architecture. Figure 3.4 presents the trust-boundary architecture visually, with components colored by their trust class.

[Figure 3.4 placeholder: trust-boundary architecture of the RFQ Copilot, showing components colored by trust class (code, language model, configuration, external service, persistence) and the flow of a turn through the eight boundaries]

Figure 3.4: Trust-boundary architecture of the RFQ Copilot, with components colored by trust class

Two Classification Sources, One Plan Factory

Every user turn enters the copilot through one of two classification sources, and both sources feed a single point of plan construction. The architecture explicitly forbids any third entry path.

The first classification source is **FastIntake**, a deterministic component that recognizes a small set of trivial messages (greetings, thanks, farewells, pure nonsense) through anchored regular expressions. FastIntake produces an `IntakeDecision` immediately, without invoking the language model. Its purpose is twofold: it eliminates the cost and latency of a language-model

call for messages that do not require interpretation, and it guarantees that the trivial cases never reach the language model where they could be over-interpreted into something they are not.

The second classification source is the **LLM Planner**, a GPT-4o invocation with a JSON-schema-enforced output. The Planner classifies the turn into one of the supported paths and proposes the relevant intent topic, target candidates, and field references. The Planner emits a `PlannerProposal`, which is then passed to the **PlannerValidator** for structural validation only. The `PlannerValidator` catches language-model failure modes such as malformed output or schema violations; it does not enforce policy and does not read the Path Registry. Its sole responsibility is to convert a raw `PlannerProposal` into a `ValidatedPlannerProposal`, which is structurally sound but not yet authorized.

Both sources feed the **ExecutionPlanFactory**, the single component in the entire system permitted to construct a `TurnExecutionPlan`. This single-construction discipline is enforced mechanically through a continuous integration check that prevents any code path other than the factory from constructing the executable plan type. The mechanism by which this check is implemented is presented in Chapter 4. The factory reads the Path Registry, applies source-aware policy enforcement (allowed and forbidden fields, intent topic existence, confidence thresholds, alias normalization), and either produces a valid `TurnExecutionPlan` or returns a structured rejection. Failures route through the Escalation Gate, which calls the same factory through a separate entry point to construct a Path 8 safe plan. There is no second path to a `TurnExecutionPlan` anywhere in the system.

The Path Registry as Policy-as-Configuration

The Path Registry is a configuration artifact that holds the entire policy surface of the copilot in a structured, declarative form. It is read at runtime only by the `ExecutionPlanFactory` and the Escalation Gate; no other component reads it, again CI-enforced.

For each combination of path and intent topic, the Path Registry defines: which intake sources are permitted to classify the turn, which evidence tools are authorized, which fields are allowed (and which are forbidden), the target resolver strategy, the access policy, the active

guardrails, the judge policy and its triggers, the working and episodic memory policy, the persistence policy, the finalizer template keys keyed by reason code, and the model profile. Every operational decision the copilot makes flows from this configuration. Adding a new path is a configuration change; it does not require modification of the pipeline stages or the factory.

The architectural significance of this design is that the policy surface of the system is inspectable, auditable, and modifiable as data rather than as code. A reviewer can read the Path Registry and know exactly what the copilot is permitted to do for any given turn. A change to the policy is a diff in a configuration file, not a change to a stage's logic.

The Pipeline and the Five Evidence Sources

A user turn that produces an answer flows through a sequence of stages between intake and persistence: target resolution, access verification (mediated through the manager API), memory load, tool execution, evidence check, context construction, response composition, deterministic guardrails, language-model judging, finalization, and persistence. Each stage owns a single decision. The complete pipeline is presented at the implementation level in Chapter 4.

The architecturally significant property of the pipeline is the limited and disciplined set of evidence sources from which the copilot is permitted to reason. Five sources are recognized.

The first is the **conversation itself**. For trivial messages classified by FastIntake (greetings and the like), and for purely conversational paths, no external evidence is required. The composition is grounded in the conversation alone. The second is the **manager service state**, the source of operational truth: lifecycle status, stage progression, attached artifacts, reminders, analytical aggregations. This is the dominant and currently-wired evidence source for the first vertical slice of the project, accessed through the manager connector that the copilot uses for all retrieval of lifecycle data. The third is the **intelligence service artifacts**: parsed MR package profiles, workbook extractions, intelligence briefings. The architecture supports this source as a first-class evidence channel, but its connector is not wired in the present implementation; integration with the intelligence service is deferred to the slices that ship Path 5 and the related intelligence-driven paths. The fourth is the **controlled domain knowledge**, a future source intended to support

questions about industry standards and norms through a bounded retrieval-augmented generation surface. The fifth is the explicit **absence of an answer**, a recognized state in which the copilot has no permitted evidence to draw on and must therefore refuse rather than approximate. The Path 8 family of templates renders the user-facing message in this case.

The discipline that follows from this enumeration is that every claim made by the copilot must be traceable to one of the first four sources, or the answer must be a Path 8 refusal. There is no fifth informal route by which the copilot might produce a claim from neither evidence nor refusal; that route would be a hallucination, and the Judge layer is its final defense.

Memory Architecture

The copilot captures bounded working memory within the active conversation thread. Working memory holds the most recent turns of the current conversation, bounded by the policy of the active path, and is persisted as part of the per-turn execution record so that the conversation history is observable and auditable across the lifetime of the thread.

The architectural posture toward longer-term memory is deliberately conservative in the present scope. Two extensions are recognized as future work and explicitly excluded from the current validated scope. The first is the injection of working memory into the Planner, Compose, and Judge prompts: the present implementation captures the conversation history but does not yet feed it back into the language-model invocations as additional context. The second is long-term episodic summarization across sessions, in which prior conversations on the same target would be condensed and made available to future turns. Both extensions are accommodated by the architecture but neither is wired in the current implementation. The bounded form delivered in the present work is sufficient for the operational paths shipped in the first vertical slice and avoids the leakage and drift risks that an under-validated memory tier would introduce.

Path Coverage in the Present Work

The architecture supports a system of seven answer paths and a Path 8 family of five protected sub-cases. Not every answer path is implemented in the present work, and the coverage is itself a deliberate consequence of the first-vertical-slice strategy explained in Chapter 4. The first vertical slice ships Path 1 (trivial conversational messages via FastIntake), Path 4 (the demo-defining operational path of evidence-grounded answers about a specific RFQ), and the full Path 8 safety infrastructure. Other answer paths short-circuit through the Escalation Gate to Path 8.1 ("not supported yet") until their own slices ship.

This coverage strategy is consistent with the trust-boundary architecture itself: shipping Path 4 and Path 8 together exercises every component of the pipeline end-to-end on real operational paths, while Path 8 ensures that every turn, including those whose answer paths are not yet ready, terminates in a bounded, deterministic, defensible response. There is no half-implemented path that returns a partially-grounded answer; either the path ships fully, or the turn routes through Path 8.

Why This Architecture Is the Innovation of the Project

The trust-bound copilot architecture is the central innovation of this work for three reasons that together distinguish it from a conventional conversational system.

The first reason is structural. The architecture commits to a strict separation of concerns between the language model and the deterministic platform: the language model produces language, the platform produces truth. This separation is enforceable at the type level (through the three-type contract that distinguishes `IntakeDecision`, `ValidatedPlannerProposal`, and `TurnExecutionPlan`) and at the build level (through CI checks that prevent any code outside the factory from constructing an executable plan). The architecture is thus self-defending in a way that policy-by-convention is not.

The second reason is operational. The architecture allocates every consequential decision to a component whose behavior is observable, auditable, and changeable independently of the

language model. The Path Registry is auditable as configuration. The Tool Executor is auditable as deterministic code. The Guardrails are testable as functions of input and output. The Judge is observable through structured logs. There is no point in the pipeline at which a black-box language-model decision determines an outcome of operational consequence.

The third reason is academic. The architecture is a defensible answer to a question the field has not yet settled: how should an LLM-driven system behave when the cost of being wrong is high? The three principal alternatives sketched at the start of this section each fail this question in a different way. The trust-bound architecture does not claim to be the only answer, but it offers a coherent answer that is implementable, testable, and inspectable, and that allocates risk to the components designed to handle it. The chapter that follows develops the implementation of this architecture, and the validation chapter proves that the eight trust boundaries are enforced not only in design but in code.

3.6 Evidence Boundary and Source-of-Truth Strategy

The trust-bound architecture presented in the previous section enumerates five evidence sources from which the copilot is permitted to reason. The architectural significance of this enumeration is not the count of sources but the discipline that governs which source applies to which turn. This section formalizes that discipline as the *evidence boundary* of the copilot, and explains the source-of-truth strategy that makes the boundary enforceable rather than aspirational.

The Evidence Boundary as an Architectural Commitment

Every claim made by the copilot is either grounded in retrievable evidence or it is not. A system that conflates these two cases produces the failure mode this work was explicitly designed to prevent: an answer that sounds confident, that is locally consistent, and that has no stable relationship to the platform's actual state. The evidence boundary is the architectural commitment that prevents this conflation. It states that the set of sources from which the copilot

may draw evidence is finite, named, and decided before the language model is invoked, not after.

The boundary operates at two levels. At the *path level*, the boundary determines which evidence sources are even legal for a given turn: an operational question about a specific RFQ may draw from manager state and intelligence artifacts but not from controlled domain knowledge; a question about an industry standard may draw from controlled domain knowledge but not from operational state; a greeting requires no evidence at all. The Path Registry encodes this correspondence statically, and the ExecutionPlanFactory enforces it at plan construction time. At the *turn level*, the boundary determines which specific instances of evidence are loaded into the language model's context: which fields, from which target, with which scoping. The per-path field whitelist and the per-target labelling discipline introduced in Section 3.5 operate at this level.

The Five Evidence Sources Revisited

The five evidence sources introduced in Section 3.5 are revisited here as the architectural primitives of the boundary, with the source-of-truth posture each one implies.

The **conversation-only source** applies when no external evidence is needed. Trivial messages classified by FastIntake (greetings, thanks, farewells) and out-of-scope refusals fall into this case. The composition draws on the user's message and the conversation context alone. There is no risk of factual hallucination because no factual claim is made.

The **manager service state** is the operational source of truth for everything the platform knows about RFQ lifecycle. Stage progression, mandatory field values, blocker status, attached artifacts, reminder definitions, analytical aggregations: all of this lives in the manager service. The copilot retrieves manager state through the same authorized API surface that the user interface consumes, with the requesting user's actor context propagated explicitly. This is the dominant evidence source for the operational paths shipped in the first vertical slice of the present work.

The **intelligence service artifacts** are the derived source of truth for the documents that surround an RFQ. The intelligence service produces deterministic profiles of the MR package and the estimation workbook, and an intelligence briefing that aggregates parsed content for downstream presentation. When a turn requires reasoning over document content (for example, a question about the BOM line items or the workbook cost breakdown), the architecture designates the intelligence artifacts as the authorized source. In the present implementation the connector that would let the copilot consume these artifacts at runtime is not wired; the intelligence service exposes its outputs to the user interface, and the copilot's consumption of intelligence artifacts is deferred to the slices that ship the corresponding paths.

The **controlled domain knowledge** is a future evidence source intended to support questions about industry standards, regulatory norms, and codified rules of the domain (ASME, TEMA, SAES, SAMSS). It is structurally a bounded retrieval-augmented generation surface: a curated corpus, a vector retrieval, and a strict guardrail that forbids answers from drifting beyond what was retrieved. It is named as an architectural primitive in this chapter because the trust-boundary architecture must accommodate it cleanly when it ships, and because its absence in the present work is itself a deliberate scope statement.

The fifth case is the **absence of permitted evidence**, which is a recognized state rather than a failure. When a turn falls into a path or intent for which no evidence is available (because the path is not yet supported, because access was denied, because the requested target does not exist, because the question is out of scope, or because the available evidence is insufficient to answer safely), the Escalation Gate routes the turn through the Path 8 family. Path 8 is not an error path: it is a structured response generator whose templates render a defensible message identifying what the copilot could not do and why. The architectural significance of Path 8 is that the absence of evidence is itself an evidence state that the system handles deliberately, rather than a degenerate case that the language model is asked to paper over. Figure 3.5 presents the evidence boundary as the decision flow that determines which source applies to a turn.

[Figure 3.5 placeholder: evidence boundary diagram showing the decision flow from turn classification to evidence source selection, with the five sources and the Path 8 family as terminal states]

Figure 3.5: The evidence boundary of the RFQ Copilot: decision flow from turn classification to evidence source selection

Why This Boundary Is Enforceable

The evidence boundary is enforceable, rather than merely declarative, because three architectural properties hold together.

The first property is the *policy-as-configuration* discipline established by the Path Registry. Every authorized evidence source for every (path, intent topic) combination is named in configuration that the ExecutionPlanFactory reads before plan construction. A turn for which no evidence source is authorized cannot be assembled into a valid plan and is routed to Path 8.

The second property is the *single-construction discipline* of the ExecutionPlanFactory. The factory is the only component permitted to assemble the executable plan, and the factory reads the Path Registry. There is no other code path through which an executable plan can be constructed without consulting the boundary. A continuous integration check enforces this discipline mechanically.

The third property is the *deterministic guardrail and Judge layer* described in Section 3.5. After the language model has composed a draft response, deterministic guardrails check that every claim made in the draft is supported by retrieved evidence, that the claim does not

drift outside the scope of the path, and that the response shape conforms to the expected format. The language model Judge then performs a final verification with explicit triggers (such as fabrication, scope drift, and forbidden inference) that would not be reliably caught by deterministic rules alone. The Judge is the last line of defense, not the first.

These three properties together make the evidence boundary something the system enforces rather than something the system documents. The defense rests on the structure of the architecture, not on the discipline of any individual component, and certainly not on the language model's compliance.

3.7 Data Model and API Contracts

This section presents the platform's data model and the API contract conventions that govern inter-service communication. The detailed entity-relationship diagram of the manager service was presented in Section 3.4. This section operates at the cross-service level: the data ownership posture, the contract conventions, and the resource families that constitute the principal API surfaces.

Data Ownership and Service Boundaries

The platform observes a strict data ownership posture, restated here in cross-service form. The **manager service** owns lifecycle data: RFQs, workflows, stage instances, subtasks, notes, files, reminders, and the operational analytics derived from them. The **intelligence service** owns analytical artifacts derived from documents: parsed MR package profiles, parsed workbook profiles, intelligence briefings, and the cross-check reports produced from them. The **copilot service** owns conversation state: threads, turns, bounded working memory, and the execution records that capture per-turn decisions for forensic and validation purposes. The **user interface service** owns presentation state only.

No service caches another service's data. When the copilot needs lifecycle state, it retrieves it from the manager at the moment of the response; it does not maintain a local replica. When

the intelligence service needs to identify the RFQ associated with a parsed package, it does so through a cross-reference key, not by holding a copy of the manager's lifecycle representation. This discipline preserves the integrity of each service's source-of-truth posture and prevents the silent drift that arises when multiple services hold copies of the same data.

API Contract Conventions

Inter-service communication uses HTTP and REST, with three conventions that hold consistently across services. Every endpoint is documented in an OpenAPI specification that is the single source of truth for its public contract. Consumers depend only on the published specification, not on implementation details. Versioning is explicit at the path level, allowing a service to evolve its internal representation without breaking its consumers.

Errors follow a consistent shape across services. A response carrying an error includes the HTTP status code, a machine-readable error type, a human-readable message, and a correlation identifier that propagates across service boundaries. The correlation identifier is what permits a single user-facing turn to be traced through every service it touches. Authorization failures and authentication failures are distinguished consistently (401 for unauthenticated requests, 403 for authenticated but unauthorized, 404 for resources the requesting user is not permitted to know exist).

Idempotency is observed where it matters. State-changing operations carry stable identifiers in their request bodies where appropriate, allowing the caller to retry on transient failures without producing duplicate state. The append-only semantics of notes, presented in Section 3.4, are an architectural defense in the same family.

A note on deployment posture closes this subsection. Each service is independently containerizable and ships with its own Docker configuration, sufficient to bring the service up in isolation or in a per-service compose scenario. A unified root compose orchestration that brings the entire platform up as a single coherent stack is recognized as a hardening direction rather than a current deliverable: the present work demonstrates the services individually and through

targeted multi-service scenarios, with full-stack orchestration deferred to a later integration pass.

Principal Resource Families

Three principal API surfaces define the inter-service contract space.

The **manager service API** exposes seven resource families: RFQ, Workflow, RFQ_Stage, Subtask, File, Note, and Reminder. Endpoints across these families cover registration, retrieval, lifecycle progression, attachment management, and analytical aggregation, with the contract evolution from the V1 baseline noted in Section 3.4. The detailed endpoint catalogue is the subject of the implementation chapter.

The **intelligence service API** exposes parsing and briefing capabilities. Its principal resources are the MR package profile, the workbook profile, and the intelligence briefing. Endpoints support both synchronous request-response retrieval (for already-parsed artifacts) and parsing triggers initiated by the manager service when an estimation workbook is uploaded. The intelligence service is read-heavy by design: it consumes documents and produces structured artifacts, but it does not own lifecycle state.

The **copilot service API** exposes conversational capabilities. Its principal resources are the thread, the turn, and the persisted execution record. The copilot does not expose its internal pipeline as separate endpoints; the pipeline is internal architecture, not contract surface. From the perspective of consuming services, the copilot's contract is a small surface centered on submitting a turn and retrieving the resulting response, with the per-turn execution record exposed for observability and validation purposes.

The contract details, including request and response schemas, error semantics, and the full endpoint catalogues, are developed in Chapter 4 alongside the implementation that delivers them.

3.8 Platform Integration: UI and Intelligence Layers

The architecture chapters so far have presented the platform service by service. This section presents the platform as a whole, focusing on how the user interface service and the intelligence service compose with the manager and copilot services to form an integrated experience. The treatment here is deliberately shorter than the depth given to the manager and copilot services in Sections 3.4 and 3.5: the integration layer is essential to turning two engines into a platform, but its complexity lies in correctness and discipline rather than in original architectural commitments.

The User Interface as the Front of the Platform

The user interface service, `rfq_ui_ms`, is implemented as a single-page application built on Next.js with TypeScript. Its responsibilities are scoped tightly: it consumes the manager service's analytical and lifecycle endpoints to render the portfolio and detail views, it presents the copilot's conversational layer as an entry point both at the portfolio level and from within an individual RFQ detail page, and it surfaces the intelligence service's outputs (parsed profiles, briefings, anomaly flags) within the RFQ detail experience.

The UI does not duplicate the platform's domain logic. It does not compute analytics that the manager service does not already expose, it does not classify or interpret user queries before submitting them to the copilot, and it does not transform or summarize intelligence service outputs beyond presentation formatting. This restraint is intentional: the same data ownership discipline that keeps service boundaries clean at the back end keeps the front end free of duplicated and potentially divergent logic. When the user interface needs information, it asks the authoritative service for it.

The Intelligence Service in the Integrated Flow

The intelligence service, `rfq_intelligence_ms`, participates in the target integrated flow at two distinct points. The first is at the moment of artifact arrival: when an MR package or

an estimation workbook is uploaded against an RFQ through the manager service, the manager triggers a parsing call to the intelligence service, which produces a deterministic profile of the artifact and persists it under the corresponding RFQ key. The second is at retrieval: when the user interface or the copilot needs to display or reason over parsed content, the intelligence service serves the persisted profile through its read API. In the present work, the parsing capabilities required by this two-moment flow are implemented at the foundation level described in Chapter 2; the trigger and retrieval surfaces are exercised through the demonstration narrative without claiming production-grade integration.

This two-moment role is what permits the intelligence service to remain read-heavy and analytical in posture, without owning lifecycle state of its own. It produces artifacts; the manager owns the lifecycle to which the artifacts are attached; the UI displays them; the copilot reasons over them under the trust-boundary discipline of Section 3.5. Each service does its single job.

The End-to-End Request Flow

A representative request flow illustrates how the four services compose. A user opens an RFQ in the user interface. The UI calls the manager service to retrieve the RFQ's lifecycle state, its attached artifacts, and its computed progress, and it calls the intelligence service to retrieve the parsed profile of the estimation workbook if one has been uploaded. Both calls return within tolerances appropriate for an interactive experience and the UI renders the detail page. The user then opens the RFQ-bound copilot panel and asks a question. The copilot accepts the turn, classifies the path, retrieves access verification and lifecycle state from the manager service through its authorized endpoints, retrieves the relevant intelligence artifacts when the path requires them, composes a draft response, verifies it through the deterministic guardrails and the Judge, and returns the finalized answer to the UI for presentation. The UI displays the answer alongside its evidence references. Figure 3.6 presents this flow at the architectural level.

[Figure 3.6 placeholder: platform integration view showing the end-to-end request flow across the user interface, manager, copilot, and intelligence services]

Figure 3.6: Platform integration view: the end-to-end request flow across the four implemented services

The architectural property that this flow illustrates is composition without coupling. Each service knows its own boundary, and the integration emerges from the disciplined use of contracts rather than from cross-service knowledge of internals. A change to the copilot’s pipeline does not propagate to the manager service. A change to the intelligence service’s parsing implementation does not propagate to the user interface. A change to the user interface’s presentation does not propagate to either backend service. The integration is what the contracts allow, and only that.

3.9 Architectural Decisions and Trade-offs

The architectural decisions presented in the preceding sections were not the only options considered. This section summarizes the major decisions made during the project as a compact reference, identifying for each one the alternatives considered, the option chosen, the rationale that drove the choice, and the principal consequence accepted with it. The full Architectural Decision Record entries, structured as described in Chapter 2, are maintained in the project’s decision register and are not reproduced in full here.

CONCEPTUAL AND TECHNICAL ARCHITECTURE

Decision	Alternatives	Chosen	Rationale	Consequence
Service decomposition	Monolith, microservices	Microservices	Clear data ownership and independent evolution of layers.	Inter-service communication overhead and contract discipline required.
Intra-service pattern	Free-form, MVC, BACAB layered	BACAB layered	Consistent structure across services, enforced separation of concerns, alignment with host firm conventions.	Higher up-front structure cost; small services may feel over-organized.
Operational source of truth	Distributed across services, owned by manager	Owned by manager	Single source-of-truth posture; copilot cannot drift from operational reality.	Manager service becomes a critical dependency for every other service.
Reminder ownership	Inside manager service, separate communication service	Inside manager service	Reminders are tightly coupled to lifecycle artifacts; extraction would require either caching lifecycle state elsewhere or per-call round trips.	Reminders may need to migrate when the communication pillar matures.
Conversational design	Generic LLM agent, naive RAG, trust-bound copilot	Trust-bound copilot	Industrial estimation requires authority over operational truth that an LLM cannot be granted.	Higher architectural complexity; more components to maintain than a generic agent would require.
Policy surface	In-code policy, policy-as-configuration	Policy-as-configuration	Auditable, modifiable as data	Configuration discipline
ENET'COM SFAX	via Path Registry		rather than as code; supports scope expansion without	becomes critical; misconfigured registry has

These eight decisions, taken together, constitute the architectural commitments of the project. Each is reversible at a cost: the cost of reverting a decision is what the consequence column estimates. The most consequential of these decisions is the trust-bound copilot design, because it determines the entire shape of the conversational layer and the discipline that governs it. The least consequential, in the sense that its reversal cost is lowest, is the in-service IAM shim, which is structurally a placeholder for a dedicated service whose extraction is anticipated rather than resisted.

3.10 CONCLUSION

This chapter has presented the conceptual and technical architecture of Itqan. It opened with the platform-level view and the four-pillar vision, distinguishing what is implemented in the present work from what remains as future scope. It introduced the microservices architecture and the BACAB layered pattern that disciplines each service internally. It developed the operational backbone in depth, presenting the manager service as a lifecycle orchestration service whose mission, domain model, stage progression mechanics, and architectural decisions together produce the structured representation that the rest of the platform depends on. It then developed the central innovation contribution of the work, the trust-bound copilot architecture, presenting the design philosophy, the positioning against alternative conversational architectures, the eight trust boundaries, the two-classification-source and single-plan-factory model, the Path Registry as policy-as-configuration, and the disciplined memory posture. It formalized the evidence boundary that governs how the copilot reasons over platform data, named the five evidence sources from which the copilot is permitted to draw, and explained the three architectural properties that make the boundary enforceable rather than declarative. It presented the data model and API contract conventions that anchor inter-service communication, the platform integration layer that composes the services into a coherent experience, and a compact register of the major architectural decisions made during the project.

The next chapter develops the implementation of this architecture. It walks through the structural realization of the BACAB pattern in code, the resources and lifecycle mechanics of

the manager service, and the per-turn pipeline of the copilot service from intake to persistence. The validation chapter that follows then proves that the eight trust boundaries are enforced not only in design, as this chapter has presented them, but in the implementation that delivers on them.

Implementation

Contents

4.1	INTRODUCTION	81
4.2	Development Environment and Toolchain	81
4.3	Implementation of rfq_manager_ms	86
4.3.1	BACAB Layered Structure in Code	86
4.3.2	RFQ Lifecycle and Stage Progression	88
4.3.3	Subtasks, Notes, Files, and Reminders	89
4.3.4	Statistics and Analytics Endpoints	90
4.3.5	Containerization and Hardening	91
4.4	Implementation of rfq_copilot_ms	92
4.4.1	Turn Processing Pipeline and First Vertical Slice	93
4.4.2	Path Registry and Planning Logic	96
4.4.3	ExecutionPlanFactory and Single-Construction Discipline	97
4.4.4	Memory, Guardrails, and Judge Layer	98
4.4.5	Escalation Gate and Safe Failure Modes	99
4.5	Platform Integration: UI and Intelligence Modules	101
4.6	CONCLUSION	106

4.1 INTRODUCTION

This chapter presents the implementation of the architecture developed in Chapter 3. Where Chapter 3 explained the architectural commitments and the reasoning behind them, this chapter shows how those commitments became running code: the structural realization of the BACAB layered pattern in each service, the resources and lifecycle mechanics of the manager service, the per-turn pipeline of the copilot service from intake to persistence, and the parsing foundation of the intelligence service. The chapter also presents the user interface that exposes the platform to its actors and the development environment that supported the work throughout the project.

The chapter is organized to match the depth distribution of Chapter 3. The two services that carry the technical weight of this work, the manager and the copilot, are developed in their own dedicated sections. The intelligence service and the user interface are developed jointly in a shorter section that reflects their foundation-level scope in the present work. The chapter is honest about the implementation status of every component it describes: capabilities that are implemented and tested are presented with strong wording, capabilities that are implemented but await full validation are presented as such, capabilities that are architecturally specified but not yet wired in code are explicitly labeled, and capabilities that are deferred to future scope are not claimed as present achievements.

The chapter opens with the development environment and toolchain that frame the work. The validation of the implementation against the requirements catalogued in Chapter 2 and against the architectural commitments of Chapter 3 is the subject of Chapter 5.

4.2 Development Environment and Toolchain

The platform is implemented across a small set of cooperating technologies, chosen for fitness to purpose, alignment with the host firm's existing engagement conventions, and operability within the constraints of a solo end-of-studies project. This section presents the toolchain

organized by the layer it serves, with versions noted where they are material to reproducibility or to understanding the implementation choices presented in subsequent sections.

The three implemented backend services are written in Python 3.11. The web framework across the services is FastAPI version 0.115, which provides asynchronous request handling, automatic OpenAPI specification generation, and Pydantic-based input and output validation. The use of FastAPI is consistent across the manager, copilot, and intelligence services and preserves the layered pattern presented in Section 3.3 of Chapter 3, in which routes parse requests, controllers orchestrate logic, and datasources mediate persistence. Object-relational mapping in the services that own a relational database is provided by SQLAlchemy version 2, with declarative models defined in the models layer of each service. Pydantic version 2 provides the validation and serialization surfaces at the route and translator layers. The combination of SQLAlchemy for persistence and Pydantic for boundary validation gives each service a consistent two-layer typing discipline: persisted entities are typed by SQLAlchemy models, and external interfaces are typed by Pydantic schemas, with the translator layer mediating between them.

The user interface is implemented in TypeScript on Next.js version 15 with React version 18. Styling is provided by Tailwind CSS, with the shadcn/ui component library used for higher-level interactive elements such as panels, modals, and form controls. The frontend's role within the platform, presented in Section 3.8 of Chapter 3, is to consume the manager service's analytical and lifecycle endpoints and to surface the copilot's conversational layer; its implementation is described in Section 4.5.

The platform's persistence layer is not uniform across services, reflecting the differing maturity and operational needs of each. The manager service uses PostgreSQL as its primary database, with the schema realizing the seven entities presented in Section 3.4 of Chapter 3. The intelligence service also uses PostgreSQL for its persisted analytical artifacts. The copilot service uses a SQLite database for its development-mode persistence of threads, turns, and execution records; this choice reflects the fact that the copilot's persistence load in the present scope is conversational rather than transactional, and the development-mode SQLite configuration is sufficient for the validation activities of Chapter 5. Migration to PostgreSQL

is straightforward thanks to the SQLAlchemy abstraction and is anticipated when the copilot's deployment context warrants it.

The copilot service's language-model invocations are made through the Azure OpenAI deployment endpoint, with the specific model accessed through deployment environment variables rather than hardcoded constants. This indirection follows two practical disciplines: it permits the deployed model to be changed without code modification, and it keeps API credentials and deployment-specific identifiers out of the version-controlled source. The trust-boundary architecture presented in Chapter 3 is independent of the specific language model deployed, although the engineering decisions developed in Section 4.4 reflect calibration against the Azure OpenAI deployment used during the project.

Each backend service is independently containerizable through its own Dockerfile. The manager and intelligence services include service-level docker-compose configurations for local and scenario-based execution, while the copilot service currently provides a Dockerfile without a dedicated compose file. A unified root docker-compose stack that brings the entire platform up as a single coherent orchestration is recognized as an integration-hardening direction rather than a current deliverable, consistent with the deployment posture noted in Section 3.7 of Chapter 3.

Source control is provided by Git, with the services organized in a polyrepo-in-monorepo structure under a shared project repository. Each service keeps its own internal structure, dependencies, tests, and configuration files, while the top-level repository provides a common workspace for development and scenario orchestration. Each service follows a consistent internal structure aligned with the BACAB layered pattern, presented in detail in the implementation sections that follow. Automated tests are present across the backend services, and the copilot service includes anti-drift tests that verify architectural invariants such as the single-construction discipline of the ExecutionPlanFactory and the Path Registry reader allow-list. At repository level, continuous integration is currently configured for the manager service, while CI coverage for the copilot, intelligence, and frontend services remains a hardening direction. Project management artifacts (the Scrumban board, the architectural decision register, the risk register, the stakeholder alignment matrix, and the sprint execution log) are maintained in the project Control Center described in Chapter 2.

A consolidated reference to the principal technologies used in the present work is presented in Table 4.1.

Layer	Technology	Role
Backend services	Python 3.11	Implementation language across the three backend services.
Backend services	FastAPI 0.115	Web framework providing asynchronous request handling and OpenAPI generation.
Backend services	SQLAlchemy 2	Object-relational mapping in the models layer of each service.
Backend services	Pydantic 2	Input and output validation at routes and translators.
Frontend	Next.js 15 / React 18	Single-page application framework.
Frontend	TypeScript	Implementation language for the user interface.
Frontend	Tailwind CSS / shadcn/ui	Styling and component library.
Data stores	PostgreSQL	Primary database for the manager and intelligence services.
Data stores	SQLite	Development-mode persistence for the copilot service.
External integrations	Azure OpenAI deployment	Language-model backend for the copilot, accessed via deployment environment variables.
Containerization	Docker	Per-service container images.
Containerization	docker-compose (manager and intelligence)	Service-level orchestration for the manager and intelligence services; no compose file for the copilot, no unified root composition.
ENET'COM SFAX Development tooling	Git	Source control across the four service repositories.

The implementation of each service against this toolchain, with the per-service configurations and the resource-level realization of the architectural commitments of Chapter 3, is the subject of the sections that follow.

4.3 Implementation of `rfq_manager_ms`

The manager service is the operational backbone of the platform, the most mature service in the present work, and the one whose implementation most directly delivers the lifecycle representation that every other service depends on. This section presents the manager service implementation in five parts: the BACAB layered structure as it is concretely realized in the manager codebase, the RFQ lifecycle and stage progression mechanics, the supporting resources for subtasks and notes and files and reminders, the statistics and analytics surface, and the containerization and operational hardening status of the service.

4.3.1 BACAB Layered Structure in Code

The manager service realizes the BACAB layered pattern presented in Section 3.3 of Chapter 3 through a consistent folder organization that mirrors the active and support layers of the pattern. Each Python module corresponds to one layer, and the layer's responsibilities are enforced by convention and by the import discipline maintained across the codebase. Figure 4.1 presents the principal folders and the role of each.

[Figure 4.1 placeholder: folder structure of the manager service showing the BACAB layered organization with active layers (routes, controllers, datasources, translators, connectors) and support layers (models, schemas, config, utils)]

Figure 4.1: Folder structure of the `rfq_manager_ms` service realizing the BACAB layered pattern

The `routes` folder contains one module per resource family, each defining the FastAPI router and the HTTP endpoint declarations. Routes parse the incoming request, validate the input shape against a Pydantic schema, delegate to the appropriate controller method, and translate the controller's return value into the HTTP response. Routes contain no business logic and never access the database directly. The `controllers` folder contains the business logic and the orchestration of operations: a controller method may invoke several datasource calls, apply business rules, and decide what data to read or write. Controllers are the layer where the lifecycle progression rules of the manager service live. The `datasources` folder contains the persistence operations: each datasource module wraps the SQLAlchemy queries for a specific entity and returns domain objects. Datasources do not make decisions; they execute reads and writes on behalf of the controller layer. The `translators` folder mediates between the SQLAlchemy models that represent persisted entities and the Pydantic schemas that cross the API boundary, preventing internal data structures from leaking into external contracts. The `connectors` folder is reserved for calls to external services; in the manager service, this layer is light because the manager does not currently consume external services in its principal flows.

The support layers complete the structure. The `models` folder contains the SQLAlchemy declarative models for the seven primary entities introduced in Section 3.4 of Chapter 3. The `schemas` folder contains the Pydantic models used at the routes and translator boundaries. The

`config` folder centralizes service configuration drawn from the environment, including the database connection string and the deployment-specific identifiers that should not appear in source. The `utils` folder collects helpers that do not belong to any single layer.

The discipline this structure enforces is straightforward: a request flows through the layers in a single direction, and a layer can only call into the layer below it. A controller may call datasources and translators; a datasource may call models; a route may call controllers and translators but not datasources or models. The continuous integration checks for the manager service include lint rules that flag layer-boundary violations.

4.3.2 RFQ Lifecycle and Stage Progression

The lifecycle representation of an RFQ is realized through three database tables and the controller methods that maintain them. The `rfq` table holds the root record with identifying metadata, classification, source channel, and overall lifecycle status. The `workflow` table holds the versioned templates that define the sequence of stages an RFQ passes through. The `rfq_stage` table holds the stage instances tied to a specific RFQ, with each instance carrying its progression state, its mandatory field values, its blocker state, and its timestamps.

When an RFQ is created through the manager service, the controller orchestrates a multi-step operation: it validates the input metadata, creates the `rfq` record, looks up the workflow template referenced by the RFQ's classification, and instantiates one `rfq_stage` record per stage in the template. The first stage of the new RFQ is marked active immediately. The overall status of the RFQ is computed from the state of its stages and is updated as the lifecycle progresses.

The most consequential controller method in the manager service is the stage advancement method, which realizes the three deterministic gates introduced in Section 3.4 of Chapter 3. The mandatory-field gate reads the stage's mandatory field list from the workflow template and verifies that each required field is populated; missing fields produce a structured error identifying them. The blocker gate reads the stage-level blocker fields (the status indicator and the reason code) and refuses the advance while the blocker state is unresolved. The transition-validity gate computes the next valid stage from the workflow template and refuses

any request whose target stage is not the computed next stage. When all three gates pass, the controller commits the advance: the current stage is marked complete with a timestamp, the next stage is activated, the RFQ's overall progress is recomputed, and the lifecycle timestamps and stage state are updated in a traceable way. Listing ?? sketches the structure of the advance-stage controller at a level of abstraction appropriate for this chapter.

```
[caption=Structure of the stage advancement controller method (illustrative)., label=lst:advancestage, basic, frame = single, breaklines = true]def advancestage(rfqid, targetstageid, payload, actor) :
stage = stagesds.get_activestage(rfqid) template = workflowsds.get_template(stage.workflowid)

Gate 1: mandatory-field gate missing = checkmandatory_fields(stage, template, payload) if missing :
raise MandatoryFieldsMissing(missing)
```

```
Gate 2: blocker gate if stage.blockersstatus == "unresolved" : raise BlockerUnresolved(stage.blocker,
```

```
Gate 3: transition-validity gate nextstage = compute_nextstage(stage, template) if nextstage.id !=
targetstageid : raise InvalidTransition(expected = nextstage.id)
```

```
Commit the advance stagesds.mark_complete(stage.id, actor) stagesds.activate(nextstage.id) rfqds.reco
```

The three gates and the commit step are presented here in their conceptual order. The actual controller method composes these operations within a single database transaction so that a partial advance never leaves the lifecycle in an inconsistent state. The traceability posture currently rests on lifecycle timestamps, persisted stage state, append-only notes, and non-destructive update semantics; a dedicated audit-history surface remains a hardening direction (FR-MGR-07 in the requirements catalogue of Chapter 2).

4.3.3 Subtasks, Notes, Files, and Reminders

Around the core lifecycle resources, the manager service implements four supporting resource families that capture the artifacts and coordination signals attached to each stage of an RFQ.

Subtasks are finer-grained units of work attached to a specific stage. Each subtask carries its own status, assignee, and due date. Subtasks support soft deletion: a deleted subtask is retained in the database with a deleted-at timestamp, and the parent stage's progress calculation

excludes soft-deleted subtasks from its denominator. This semantics, named in FR-MGR-08, prevents progress regressions when subtasks are removed and preserves the audit value of the deletion event.

Notes are append-only textual artifacts attached to a stage. The notes endpoint exposes only POST and GET; no PUT or DELETE is offered. This is an architectural defense, not a missing feature: the immutability of notes preserves the integrity of the communication trail attached to each stage and supports the audit posture established in Section 3.7 of Chapter 3.

Files are persisted documents attached to a stage. The file model distinguishes file types in its schema, including a designated type for the estimation workbook. The upload endpoint accepts the file payload, stores it in the configured file storage backend, and creates the database record linking the file to its stage. In the present implementation, downstream parsing of the estimation workbook by the intelligence service is available through manual and direct trigger paths. The automatic event-driven cascade from manager upload to intelligence parsing is architecturally prepared but not yet wired as an autonomous event-bus consumer, consistent with the posture established in Section 3.3 of Chapter 3.

Reminders are time-anchored notifications associated with a stage or a subtask. The reminder model supports two creation modes: manual reminders set by a user with an explicit due date and assignee, and rule-based reminders generated by the manager service in response to lifecycle conditions (for example, a reminder generated when a stage approaches a deadline). The reminder dispatch mechanism in the present work is log-only: when a reminder fires, the event is recorded in the service log with the reminder identifier, the firing time, and the assigned actor, but no outbound message is sent to a real communication channel. The wiring of reminders to actual delivery channels (email, in-application notifications) is part of the future communication service scope discussed in Section 3.9 of Chapter 3.

4.3.4 Statistics and Analytics Endpoints

The manager service exposes an analytical surface that aggregates RFQ data across operationally meaningful dimensions. The implemented analytical endpoints serve real database queries

against the persisted lifecycle records and are sufficient for the demonstration narrative of Chapter 5.

The currently-validated analytical capabilities are those that depend only on data that the manager service already owns. The **pipeline aggregation** endpoint groups RFQs by status, classification, and stage, producing the counts that the user interface uses to render its dashboard cards. The **win-rate aggregation** endpoint computes the win, loss, and pending counts across the RFQs that have reached a terminal status, with optional grouping by client or by workflow template. The **by-client aggregation** endpoint produces per-client portfolio summaries, including total RFQ count and current pipeline value where pricing has been recorded.

Two analytical capabilities described in the four-pillar vision of Chapter 1 are deliberately not claimed as currently implemented. The **margin analysis** endpoint, which would compare bid prices to actual fabrication costs, depends on outcome data that the manager service does not currently capture and is deferred to future scope. The **estimation accuracy** endpoint, which would compare initial estimates to final billed values, depends on the same kind of outcome data and is also deferred. Both capabilities are part of the broader Pillar 3 (Executive Intelligence and BI) and Pillar 4 (Historical Learning and Prediction) maturation paths.

4.3.5 Containerization and Hardening

The manager service ships with a Dockerfile that produces a self-contained image with the Python runtime, the application code, and the FastAPI server. A service-level docker-compose configuration brings the manager service up alongside its PostgreSQL dependency for local development and demonstration scenarios. As noted in Section 4.2 and in the deployment posture of Chapter 3, no unified root docker-compose orchestration is provided in the current scope; per-service composition is the operational mode.

Several hardening passes have been applied to the manager service over the course of development. Structured logging with correlation identifiers is enabled across the request path, allowing a single user-facing operation to be traced from the route through the controllers and datasources. The service exposes a readiness probe and a metrics endpoint compatible with

standard orchestration and observability tooling, satisfying NFR-OPS-02 from the requirements catalogue. Database migrations are managed through Alembic, with the schema evolution captured in versioned migration files committed alongside the application code. Input validation at the route layer rejects malformed payloads before they reach the controllers, and the controllers raise typed exceptions that the route layer translates into structured HTTP error responses with stable error type codes. The full Postman collection that exercises the manager service end-to-end is maintained as part of the validation evidence presented in Chapter 5.

A dedicated audit-log table covering every state-changing operation is acknowledged but not implemented in the current scope; the lifecycle traceability that the manager service provides today rests on timestamps, the append-only status history, and the non-destructive update semantics of the supporting resources, as established in Section 3.4 of Chapter 3 and FR-MGR-07 of Chapter 2.

4.4 Implementation of `rfq_copilot_ms`

The copilot service is the principal innovation contribution of this work, and its implementation is the most architecturally consequential part of the chapter. This section presents the copilot's implementation in five parts: the turn processing pipeline and the first vertical slice that the implementation delivers, the Path Registry and the planning logic that classifies user turns, the `ExecutionPlanFactory` and the single-construction discipline that anchors the trust-boundary architecture, the memory and guardrails and Judge layer that compose the response under deterministic supervision, and the Escalation Gate that handles failure modes safely. The implementation realizes the architecture presented in Section 3.5 of Chapter 3, with the scope honesty established there carried forward into this chapter without modification.

The copilot codebase has evolved through two implementation lanes during the project, named v1 and v2. The v1 lane established the basic conversational surface with a minimal request-response endpoint, sufficient to validate the integration of the language model and to anchor the early demonstration narrative. The v2 lane is the trust-boundary implementation: it is the lane that delivers the `FastIntake/Planner` classification, the `PlannerValidator`, the `ExecutionPlanFactory`,

the Path Registry, the Escalation Gate, and the full pipeline of deterministic stages presented in Chapter 3. The v1 lane is preserved in the codebase to support legacy demonstration paths that the user interface still consumes for thread lifecycle operations, but new architectural development happens entirely in the v2 lane. References to “the copilot” in the remainder of this section refer to the v2 implementation unless otherwise noted.

4.4.1 Turn Processing Pipeline and First Vertical Slice

The turn processing pipeline is the spine of the copilot implementation. A user turn enters the service through a single API endpoint that accepts the user’s message, the active thread identifier, the actor context, and the conversational mode (portfolio or RFQ-bound). The turn flows through a fixed sequence of stages between intake and persistence, with each stage owning one decision and reporting structured success or failure to the stage that follows. Figure 4.2 presents the pipeline as it is realized in the v2 implementation.

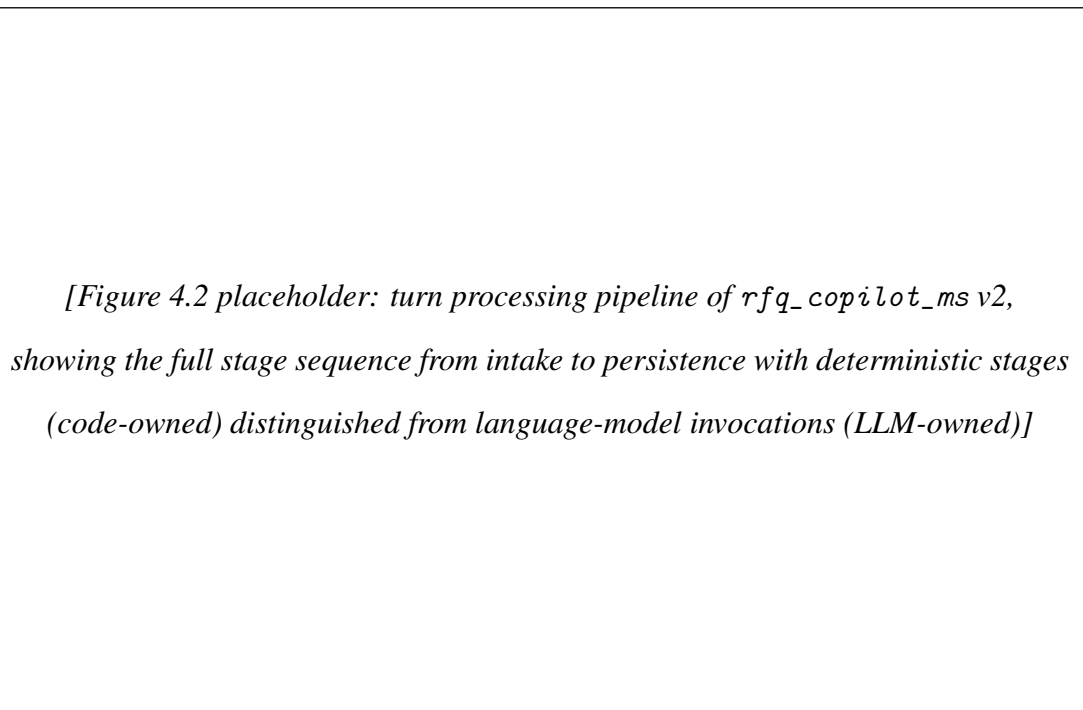


Figure 4.2: Turn processing pipeline of `rfq_copilot_ms` (v2 implementation), with deterministic stages distinguished from language-model invocations

The pipeline begins with **FastIntake**, a deterministic classifier that matches the incoming user message against a small set of anchored regular expression patterns covering trivial messages:

greetings, thanks, farewells, and pure-nonsense inputs. When `FastIntake` matches, it produces an `IntakeDecision` immediately and the language-model `Planner` is never invoked. When `FastIntake` does not match, the turn flows to the LLM **Planner**, a structured invocation of the configured Azure OpenAI deployment with a JSON-schema-enforced output. The `Planner` classifies the turn into a path, proposes the relevant intent topic, and emits a `PlannerProposal`.

The **PlannerValidator** accepts the `PlannerProposal` and performs structural validation only: it verifies the JSON schema, checks that the emitted path is one the validator recognizes, and rejects malformed outputs. The validator does not enforce policy and does not read the Path Registry. It produces either a `ValidatedPlannerProposal` or a structured trigger that the Escalation Gate handles.

Both the `IntakeDecision` from `FastIntake` and the `ValidatedPlannerProposal` from the `Planner+Validator` pair feed the **ExecutionPlanFactory**, the single component permitted to construct an executable `TurnExecutionPlan`. The factory reads the Path Registry, applies source-aware policy enforcement, and produces either a valid plan or a structured rejection that routes through the Escalation Gate.

A valid plan flows through the deterministic execution stages. The **Resolver** converts the planner's target candidates into resolved targets. The **Access** stage verifies that the requesting actor has permission to read each resolved target by calling the manager service through the manager connector; an access denial routes through the Escalation Gate with a Path 8.4 reason code. The **Memory Load** stage retrieves the bounded working memory for the active thread; its capture-only role in the present implementation is detailed in subsection 4.4.4. The **Tool Executor** invokes the tools that the Path Registry mapped from the active path and intent topic, with the deterministic invocation receiving the resolved targets and the planner-extracted fields as input. The **Evidence Check** stage verifies that the tool execution produced evidence sufficient for composition; an empty evidence result routes through the Escalation Gate with a Path 8.5 reason code.

When evidence is present, the **Context Builder** assembles the prompt for the language model from the per-path field whitelist and the per-target labelled evidence packets. For Path 4

(the demonstration-defining operational path), the implementation also provides a deterministic Path 4 renderer that produces the response without invoking the language model when the evidence and the path semantics permit it, reserving the language model invocation for cases where natural-language phrasing materially improves the answer. The **Compose** stage, when invoked, calls the language model to produce a draft response from the assembled context. The **Guardrails** stage applies a sequence of deterministic checks to the draft: evidence grounding, scope adherence, response shape conformance. The **Judge** stage performs a final language-model verification against explicit triggers (fabrication, scope drift, forbidden inference). The **Finalizer** renders the user-facing response from the draft and the verdict, and the **Persist** stage writes the per-turn execution record alongside the thread and turn updates.

The first vertical slice of the v2 implementation ships three of the eight architecturally specified paths end-to-end. **Path 1** handles trivial conversational messages classified by FastIntake; its implementation reuses the Planner-validator-finalizer infrastructure with a minimal template set and bypasses the Tool Executor and Judge stages because no factual claim is being made. **Path 4** is the demonstration-defining operational path: an evidence-grounded answer about a specific RFQ, drawing manager service state through the manager connector. **Path 8** is the safety infrastructure: its five sub-cases (8.1 unsupported, 8.2 out-of-scope, 8.3 ambiguous target, 8.4 access denied or target missing, 8.5 insufficient evidence) ship together to guarantee that every turn terminates in a bounded, deterministic, defensible response regardless of which path the turn was classified into.

Paths 2, 3, 5, 6, and 7 are architecturally specified in the Path Registry and routed through the Escalation Gate to Path 8.1 in the present implementation. This is not a degraded behaviour: it is the architecturally correct response when a path is recognized but its slice has not yet shipped. The deferred paths and their planned slice ordering are discussed in Chapter 5 as part of the path coverage validation.

4.4.2 Path Registry and Planning Logic

The Path Registry is implemented as a structured Python module, organized as a configuration artifact rather than as runtime logic. For each combination of path identifier and intent topic, the registry declares the allowed intake sources, the evidence tools, the resolver strategy, the access policy, the field whitelist, the field aliases, the confidence threshold, the active guardrails, the judge policy and triggers, the memory policy, the persistence policy, and the finalizer template keys keyed by reason code. The registry is the single source of policy truth for the copilot, and the runtime contract is that only the ExecutionPlanFactory and the Escalation Gate are permitted to read it. This restriction is enforced by an automated check described in subsection 4.4.3.

The planning logic that produces a PlannerProposal from a user message is implemented as a structured prompt to the Azure OpenAI deployment, with the response shape constrained by a JSON schema. The prompt instructs the model to classify the turn into one of the recognized paths, to identify the intent topic, to extract any explicit target references, and to surface the field references the user is asking about. The schema enforcement is what distinguishes this Planner invocation from a free-form language-model call: the Planner's output is structured by construction, and any malformed output is rejected by the PlannerValidator before it can influence the rest of the pipeline.

The PlannerValidator is implemented as a pipeline component of the copilot service and performs three classes of structural check. The first is JSON schema conformance: the validator verifies that the proposal contains the required fields and that each field is of the expected type. The second is enumeration validation: the validator checks that the path identifier is one the system recognizes and that the intent topic is one declared in the registry's index. The third is the absence of LLM-specific failure modes: empty fields, hallucinated path identifiers, contradictory metadata. The validator does not enforce policy; it does not check whether the proposed path is allowed for the actor, whether the requested fields are in the path's whitelist, or whether the proposal would assemble into a valid plan. Those checks belong to the ExecutionPlanFactory and are presented in subsection 4.4.3.

4.4.3 ExecutionPlanFactory and Single-Construction Discipline

The `ExecutionPlanFactory` is the architectural chokepoint of the copilot service. It is the single component permitted to construct a `TurnExecutionPlan`, the typed object that flows through the deterministic execution stages of the pipeline. The factory accepts three classes of input: an `IntakeDecision` from `FastIntake`, a `ValidatedPlannerProposal` from the `Planner+Validator` pair, or an `EscalationRequest` from the `Escalation Gate`. For each class of input, the factory exposes a distinct entry point, and the source of the resulting plan is recorded in the plan's `source` field for downstream observability.

Within the factory, the path identifier and intent topic from the input are looked up in the `Path Registry`, and the policy attributes for that combination are applied to construct the executable plan. Source-aware policy enforcement applies a sequence of checks: the intent topic must exist in the registry, the proposed source (`FastIntake`, `Planner`, or `Escalation`) must be among the allowed intake sources for the (path, intent_topic) combination, the requested fields must be in the path's allowed-field whitelist and must not appear in the forbidden-field list, the confidence threshold must be met for `Planner`-sourced proposals, and the field aliases must be normalized before they are written into the plan. A failure at any of these checks produces a structured `FactoryRejection` that the `Escalation Gate` intercepts.

The single-construction discipline is enforced by an automated check that runs in continuous integration on every commit to the copilot repository. The check inspects the codebase for any direct construction of the `TurnExecutionPlan` type outside the factory module and fails the build if any such construction is detected. A complementary check enforces the registry-reader allow-list: only the `ExecutionPlanFactory` module and the `Escalation Gate` module are permitted to import from the `Path Registry` module, and any other import is flagged. A third check enforces the `FastIntake` path-range invariant: the `FastIntake` module is restricted to emitting decisions for the trivial-conversational and out-of-scope path identifiers, preventing an inadvertent extension of `FastIntake`'s classification scope from silently bypassing the `Planner`. Listing ?? sketches the public surface of the factory at the level of abstraction appropriate for this chapter.

[caption=Public surface of the ExecutionPlanFactory (illustrative)., label=lst:factory_{signature}, basicstyle, frame = single, breaklines = true]class ExecutionPlanFactory : "" Singlepermitted constructor of T

```
def buildfromintake(self, decision : IntakeDecision, actor : ActorContext, session :
SessionState) -> TurnExecutionPlan | FactoryRejection : ...
```

```
def buildfromplanner(self, proposal : ValidatedPlannerProposal, actor : ActorContext, session :
SessionState) -> TurnExecutionPlan | FactoryRejection : ...
```

```
def buildfromescalation(self, request : EscalationRequest, actor : ActorContext, session :
SessionState) -> TurnExecutionPlan : ...
```

```
Internal: reads the Path Registry and applies source-aware policy. def resolvepolicy(self, path :
PathId, intenttopic : IntentTopic, source : IntakeSource) -> PathPolicy : ...
```

The three public entry points correspond to the three classes of input the factory accepts. The escalation entry point is distinguished by the fact that it cannot fail: by construction, an escalation always produces a Path 8 plan, because the Escalation Gate has already mapped the failure to a Path 8 sub-case and selected the appropriate finalizer template. The other two entry points may return a FactoryRejection, which the Escalation Gate then routes back through the escalation entry point to construct the safe Path 8 plan.

4.4.4 Memory, Guardrails, and Judge Layer

The composition stages of the pipeline are organized around three architectural commitments: bounded memory, deterministic guardrails, and a final language-model Judge. Each is implemented to honour the scope discipline established in Section 3.5 of Chapter 3.

The **memory** layer in the present implementation is capture-only. Each turn's working memory is persisted as part of the per-turn execution record, with the most recent turns of the active thread accessible for inspection and audit. This capture is what gives the copilot its observability story: the conversation history is structured, queryable, and forensically complete. The two extensions identified in Chapter 3 are explicitly not implemented in the present scope. The first is the injection of working memory into the Planner, Compose, and Judge prompts;

the present implementation captures the conversation history but does not yet feed it back into the language-model invocations as additional context. The second is long-term episodic summarization across sessions. Both extensions are accommodated by the architecture and are planned as future work; their absence in the present implementation means that the copilot's behaviour on a given turn is determined by that turn's content and by the deterministic policy of the active path, not by accumulated conversational context.

The **guardrails** layer applies a sequence of deterministic checks to the draft response produced by the Compose stage (or to the response produced directly by the Path 4 deterministic renderer when the language model is bypassed). The evidence guardrail checks that factual statements in the draft remain consistent with the retrieved evidence packets and with the active path policy. The scope guardrail verifies that the draft does not introduce content outside the active path's declared scope. The shape guardrail verifies that the response conforms to the structural contract for the active path: required sections present, length within bounds, formatting valid. Each guardrail runs deterministically and produces a structured pass-or-fail result; a failure routes through the Escalation Gate to a Path 8 finalization with the appropriate reason code.

The **Judge** layer is the final line of defense. It is implemented as a second language-model invocation against the configured Azure OpenAI deployment, with a JSON-schema-enforced output that classifies the draft response against explicit triggers: fabrication (the draft makes a factual claim not supported by retrieved evidence), scope drift (the draft addresses a question other than the one asked), and forbidden inference (the draft draws an inference that the path's policy explicitly disallows). The Judge is the architectural equivalent of a code review for the language model's output: it does not produce content, it inspects content. A negative verdict routes through the Escalation Gate to a Path 8 finalization. A positive verdict permits the Finalizer to render the user-facing response from the validated draft.

4.4.5 Escalation Gate and Safe Failure Modes

The Escalation Gate is the single deterministic intercept for every failure trigger across the pipeline. Its implementation is structurally simple by design: it accepts a failure trigger from

any pipeline stage, maps the trigger to a Path 8 sub-case according to a deterministic rule table, constructs a structured `EscalationRequest` carrying the original trigger, the mapped sub-case, the reason code, and any context the failed stage produced, and re-enters the `ExecutionPlanFactory` through the escalation entry point to construct the `Path 8 TurnExecutionPlan`.

The mapping from failure trigger to Path 8 sub-case is the core of the gate's logic. A target resolution failure or an access denial maps to Path 8.4 (target unreachable). A confidence threshold failure or a missing intent topic maps to Path 8.1 (unsupported). An out-of-domain classification from the Planner maps to Path 8.2 (out-of-scope). An ambiguous target reference maps to Path 8.3. An empty evidence result, a guardrail failure, or a negative Judge verdict maps to Path 8.5 (insufficient evidence or unsafe answer). Each mapping is encoded as a deterministic rule in the Escalation Gate module and is the single place in the codebase where these mappings live; no other component decides which Path 8 sub-case applies.

The architectural significance of the Escalation Gate is that it converts every failure mode in the pipeline into a structured, finalized response rather than into an exception or a degraded answer. The Finalizer always renders from a `TurnExecutionPlan`, regardless of whether that plan came from FastIntake, the Planner, or an escalation. There is no parallel error path through which a partially-formed response or an unstructured exception could reach the user. The user always receives either a grounded answer or a Path 8 message identifying what the copilot could not do and why; the architectural property that makes this guarantee enforceable is the single-construction discipline of the `ExecutionPlanFactory` enforced by the continuous integration check described in subsection 4.4.3.

The implementation of the copilot service across these five aspects realizes the trust-bound architecture of Chapter 3 in code. The validation activities of Chapter 5 demonstrate that the eight trust boundaries hold not only by construction but in measured behaviour against the path coverage scenarios and the targeted adversarial probes designed to stress the boundary enforcement.

4.5 Platform Integration: UI and Intelligence Modules

The user interface and the intelligence service are presented together in this section because both serve integration-level roles in the present work rather than carrying the technical centerpiece weight given to the manager and copilot services. The user interface composes the platform's services into a usable surface for the actors identified in Chapter 2; the intelligence service delivers the deterministic parsing foundation on which the platform's analytical capabilities will eventually rest. This section presents the implementation status of each, with the scope honesty required to position both within the realistic maturity of the current work.

Intelligence Service Implementation

The intelligence service, `rfq_intelligence_ms`, is implemented as a FastAPI service following the same BACAB layered structure described for the manager service in Section 4.3. Its persistence layer uses PostgreSQL, and its principal output artifacts (parsed MR package profiles, parsed workbook profiles, intelligence briefings, cross-check reports) are persisted under the corresponding RFQ key for retrieval by downstream consumers.

The implemented parsing capabilities cover two artifact families. The **MR package parser** consumes the structured Material Requisition packages produced by GHI's clients, exemplified by the golden reference package SA-AYPP-6-MR-022 introduced in Chapter 1. The parser identifies the package's sixteen-section layout, extracts the bill of materials, the restricted vendor list, the applicable standards references, and the technical specification metadata, and produces a deterministic profile of the package as its principal output. The structural assumptions on which the parser rests are validated against the single golden reference package available during the project; broader validation against additional packages is a hardening direction explicitly acknowledged in Chapter 5.

The **workbook parser** consumes estimation workbooks structured according to the thirty-six-sheet template described in Chapter 1, exemplified by the reference workbook IF-25144.xls. The parser extracts identity metadata, item line entries from the bill of materials sheet, cost-breakdown

values from the bid summary and top sheet, and schedule profiles from the cash-flow sheets. Both parsers expose their outputs through synchronous read endpoints, which downstream consumers (the manager service for trigger-driven flows, the user interface for direct presentation) invoke when the parsed artifacts are needed.

A cross-check and anomaly surfacing capability complements the parsers. When the workbook parser encounters values that fail internal consistency checks (for example, a unit mismatch between a BOM line and its corresponding workbook entry, or a cost-per-ton value that falls outside expected bounds), the parser produces a structured anomaly flag rather than silently accepting the value or substituting a corrected one. The anomaly flags are persisted alongside the parsed profile and are retrievable through a dedicated endpoint, supporting the requirement that review-worthy artifacts not be silently accepted (FR-INT-04 in the requirements catalogue of Chapter 2).

Several aspects of the intelligence service are explicitly partial in the present scope and are flagged here for accurate framing. The **briefing generation** surface is implemented at the level required to support the demonstration narrative: it aggregates parsed content from the MR package and the workbook into a structured briefing document, but the richer cross-artifact synthesis envisioned for the long-term Pillar 4 maturation is deferred. The **intake pipeline** that would automate the parsing trigger from upstream artifact uploads is implemented through manual and direct trigger paths, as established in Section 3.3 of Chapter 3 and Section 4.3 of this chapter; the autonomous event-bus consumer that would let the intelligence service react to lifecycle events without explicit invocation is architecturally specified but not wired in the current implementation. The **manager integration** is read-only and trigger-based: the intelligence service does not modify lifecycle state and does not initiate flows in the manager service. The **copilot integration** is not implemented in the present work; the connector that would let the copilot consume intelligence artifacts at runtime is deferred to the slices that ship Path 5 and the related intelligence-driven paths, as noted in Chapter 3. The **predictive intelligence** capabilities sketched by Pillar 4 in the four-pillar vision are explicitly future scope and are not present in the implemented service. With these scope statements in place, the

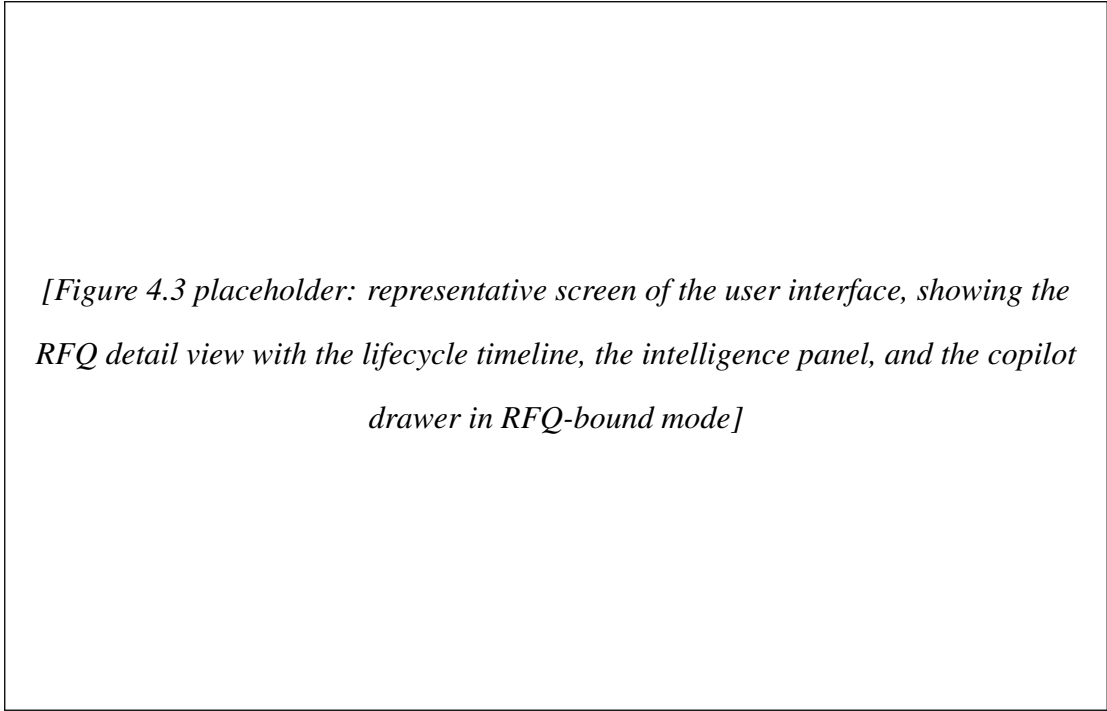
intelligence service is accurately positioned as the parsing foundation of the future intelligence layer rather than as a fully-realized analytical engine.

User Interface Implementation

The user interface, `rfq_ui_ms`, is implemented as a Next.js 15 application with React 18 and TypeScript, styled with Tailwind CSS and the `shadcn/ui` component library. Its role within the platform is to compose the manager service's lifecycle and analytical endpoints, the intelligence service's parsed artifacts, and the copilot's conversational layer into a single usable surface for the actors identified in Chapter 2.

The principal screens implemented in the user interface span the operational lifecycle of an RFQ. The **RFQ list and portfolio view** presents the full set of RFQs accessible to the current actor, with filtering and sorting by status, classification, stage, and other operationally meaningful dimensions. The **RFQ detail view** presents a single RFQ's full lifecycle representation, including its stage timeline with progression history, its attached subtasks and notes and files, its associated reminders, and an intelligence panel that surfaces the parsed artifacts produced by the intelligence service when they are available. The **dashboard** provides the executive-level view of the portfolio through key performance indicator cards drawn from the manager service's analytical endpoints, including pipeline value by status, win-rate counts, and per-client portfolio summaries.

The conversational layer is exposed through two complementary entry points consistent with the dual-mode design established in Section 3.5 of Chapter 3. The **copilot drawer** attached to the RFQ detail view provides the RFQ-bound conversational mode, in which queries are scoped to the currently-open RFQ. The **copilot page** accessible from the application shell provides the portfolio-level mode, in which queries operate over the broader portfolio context. Figure 4.3 presents a representative screen of the user interface showing the RFQ detail view with the copilot drawer open.



[Figure 4.3 placeholder: representative screen of the user interface, showing the RFQ detail view with the lifecycle timeline, the intelligence panel, and the copilot drawer in RFQ-bound mode]

Figure 4.3: Representative screen of `rfq_ui_ms`: RFQ detail view with the copilot drawer open in RFQ-bound mode

A development-mode capability supports the work of building and demonstrating the platform under variable backend availability. The frontend supports both live API consumption and demonstration mode through local fixtures, controlled through a configuration switch. Live API mode is used for integration-oriented development and for the validation activities of Chapter 5, while mock mode supports isolated UI development and demonstration resilience when one or more backend services are not running.

Several aspects of the user interface are explicitly partial or deferred in the present scope and are flagged here for accuracy. There is **no production login flow**: the actor context that the frontend submits with each request is selected client-side through a development-mode role selector rather than through an authenticated session, consistent with the IAM scope discussion in Section 3.9 of Chapter 3. The user interface is not currently **containerized**; deployment in the present work is through the development server, with containerization recognized as an integration-hardening direction. **Browser-level end-to-end tests** are not present in the current scope; testing of the integrated platform is currently performed through the manager service's Postman collection, the copilot service's pytest suites, and manual demonstration walkthroughs.

Finally, the frontend's **thread management for the copilot** currently consumes the v1 lane of the copilot service for thread lifecycle operations (creating threads, listing prior threads), while the v2 turn endpoint with its full trust-boundary pipeline serves the actual conversational exchanges; the migration of frontend thread lifecycle to the v2 lane is a recognized alignment task and is part of the future scope of the integration. With these scope statements in place, the user interface is accurately positioned as the integration and presentation surface of the platform with the integration maturity required for the validation activities of Chapter 5, rather than as a production-grade application ready for end-user deployment.

Cross-Service Integration Status

Two cross-service integration paths matter for the validation activities of Chapter 5 and are summarized here as the closing of this section. The **manager-to-intelligence path**, in which an estimation workbook uploaded to the manager service is parsed by the intelligence service and its profile retrieved by downstream consumers, is implemented through the manual and direct trigger flows discussed earlier in this chapter; an end-to-end demonstration of the path is achievable but the autonomous event-driven realization remains a hardening direction. The **copilot-to-evidence path**, in which the copilot retrieves operational evidence to ground a Path 4 answer, is implemented and exercised end-to-end against the manager service's authorized endpoints; the symmetric path that would let the copilot retrieve evidence from the intelligence service is architecturally specified but not wired in the current implementation, as established for the Path 5 family in Section 3.5 of Chapter 3. With the implementation status of the four services and their integration paths now presented, the chapter concludes by handing off to the validation activities that exercise this implementation against the requirements and architectural commitments established in earlier chapters.

4.6 CONCLUSION

This chapter has presented the implementation of the architecture developed in Chapter 3. It opened with the development environment and toolchain that frame the work, including the Python and FastAPI backend stack, the Next.js frontend stack, the per-service persistence configuration, the Azure OpenAI deployment used by the copilot service, and the per-service containerization posture. It developed the manager service implementation in depth, presenting the BACAB layered structure as concretely realized in the manager codebase, the lifecycle and stage progression mechanics through which the three deterministic gates of the architecture become guarded code, the supporting resources for subtasks and notes and files and reminders, the analytical endpoints that serve real database aggregations within the bounds the present scope supports, and the containerization and hardening status of the service. It then developed the copilot service implementation, the principal innovation contribution of the work, presenting the v1 and v2 implementation lanes, the turn processing pipeline as the spine of the v2 implementation, the Path Registry and the Planner classification logic, the ExecutionPlanFactory and the single-construction discipline enforced by continuous integration checks, the memory and guardrails and Judge layers under explicit scope discipline, and the Escalation Gate as the single deterministic intercept for failure modes. It closed with the platform integration layer, presenting the intelligence service as the parsing foundation of the future intelligence layer and the user interface as the composing presentation surface, with the scope honesty required for both to be accurately positioned within the maturity of the present work.

The next chapter validates this implementation against the requirements catalogued in Chapter 2 and the architectural commitments developed in Chapter 3, presents the formal metrics and the golden scenarios that exercise the platform end-to-end, demonstrates that the eight trust boundaries of the copilot architecture hold in measured behaviour, and discusses the business impact of the implemented capabilities against the audit pain points identified in Chapter 1.

Validation, Results, and Discussion

Contents

5.1	INTRODUCTION	108
5.2	Backend Validation of rfq_manager_ms	110
5.2.1	Endpoint Coverage and Response Validation	111
5.2.2	Workflow and Data Consistency	112
5.3	Copilot Validation: Formal Metrics	115
5.4	Copilot Validation: Golden Scenarios	118
5.5	Integration Validation	119
5.5.1	Integration Paths and Validation Status	120
5.5.2	Demonstration-Oriented Evidence	122
5.5.3	Intelligence Service Validation in the Integrated Context	122
5.5.4	Summary of Integration Validation	123
5.6	Business Impact Against Audit Pain Points	124
5.7	Limitations and Honest Discussion	127
5.8	Future Work	129
5.9	CONCLUSION	131

5.1 INTRODUCTION

This chapter validates the implementation of the Itqan platform against the requirements catalogued in Chapter 2 and the architectural commitments established in Chapter 3. It presents the evidence that the platform's claims hold in measured behaviour, discusses the business impact of the implemented capabilities against the audit pain points identified in Chapter 1, and offers an honest discussion of the limits of the present work and the direction in which the platform should evolve.

The validation methodology adopted in this chapter rests on five complementary classes of evidence. The first is service-level automated test coverage: pytest suites that exercise the public surface and the internal logic of each backend service in isolation. The second is architectural invariant verification: anti-drift tests that protect the trust-boundary architecture of the copilot service against silent regressions in the discipline that distinguishes it from a generic conversational system. The third is scenario-level evidence: smoke and pipeline tests that walk representative user interactions end-to-end through the copilot's full pipeline, including the safety paths that handle unsupported, out-of-scope, and evidence-insufficient queries. The fourth is integration evidence: cross-service scripts and demonstration walkthroughs that exercise the platform as a composed system rather than as isolated services. The fifth is business-impact analysis: a structured mapping from the implemented capabilities to the twelve audit pain points presented in Chapter 1, demonstrating which pain points the platform addresses and to what degree.

The chapter is organized to present these evidence classes in sequence, with the strongest validation areas treated first. Section 5.2 presents the validation of the manager service, the operational backbone of the platform, through its pytest suite and its associated walkthroughs. Sections 5.3 through 5.5 present the validation of the copilot service: formal metrics from the pytest suite, scenario coverage through smoke and pipeline tests, and the trust-boundary verification that anchors the innovation contribution of the work. Section 5.6 presents the integration validation across services, with the scope honesty that the foundation-level maturity of the user interface and the intelligence service requires. Section 5.7 develops the business-impact

analysis. Section 5.8 offers an honest discussion of the limits of the validation evidence itself, and Section 5.9 sets out the future work that follows from these limits. The chapter closes with a brief conclusion that returns to the project as a whole.

The validation methodology has explicit limits that are stated here at the outset rather than discovered as the chapter unfolds. Continuous integration coverage at the repository level is currently configured for the manager service only; the copilot, intelligence, and user interface services have working test suites but no automated CI workflow as of the present writing. Manager service tests are executed against a SQLite quality-gate target rather than against a PostgreSQL instance; PostgreSQL-specific validation remains a hardening direction. The copilot's language-model stages are validated through a deterministic test double rather than through live invocations of the configured Azure OpenAI deployment; live-model regression validation is recognized as a future addition. Browser-driven end-to-end tests of the user interface are not present in the current scope, and integration evidence for the user interface is collected through manual demonstration walkthroughs rather than automated browser tests. Each of these limits is revisited in the relevant section below and consolidated in the limitations discussion of Section 5.8. A consolidated summary of the validation evidence assembled across the chapter's technical sections is presented in Table 5.1. The summary is intended as a high-level reference; the detailed evidence and the methodology behind each row are developed in the corresponding sections that follow.

Area	Evidence	Result	Caveat
Manager service	259 pytest tests across 27 test files	All passed	SQLite quality-gate target; no recorded Postman execution.
Copilot service	636 pytest tests across 47 test files	All passed	Language-model stages exercised through deterministic test double; no live-model regression.
Intelligence service	118 pytest tests across 39 test files	64 passed, 48 fixture-dependent, 6 skipped	Fixture-dependent tests not reproduced in audited checkout.
Frontend	12 source-contract scripts	11 passed, 1 assertion drift	No browser-level end-to-end testing in current scope.
Integration	Scenario stack script, smoke demonstration document, seed manifests	Manual demonstration	No automated full-stack end-to-end runs.

Table 5.1: Consolidated summary of validation evidence assembled across the chapter

5.2 Backend Validation of `rfq_manager_ms`

The manager service is the operational backbone of the platform and the most mature implementation in the present work. Its validation rests primarily on a pytest suite of **259 tests across 27 test files**, all of which pass against the SQLite quality-gate target used for the

development and validation runs of the project. The full suite is executed through a single command invocation and completes in under thirty seconds on the development environment. A representative invocation and its outcome are summarized in Listing ??.

```
[caption=Manager service pytest invocation and outcome., label=lst:manager_tests, basicstyle =
, frame = single, breaklines = true] cd microservices/rfq_manager_ms DATABASE_URL =
sqlite : ///./qg.dbPYTHONPATH = .python - mpytest - q - -tb = no

259 passed, 121 warnings in 25.29s
```

The **121 warnings** reported in the run are deprecation and configuration warnings that do not indicate test-level failures; the suite itself passes in full. The validation is qualified by two scope statements that are made explicit here. First, the test suite is executed against SQLite rather than PostgreSQL: PostgreSQL is the production database target for the manager service, and the suite is structured so that PostgreSQL validation is feasible, but PostgreSQL-specific validation is not part of the present evidence and remains a hardening direction. Second, a Postman collection covering the manager service end-to-end exists in the project repository (docs/postman/rfq_manager_ms_postman_full_walkthrough_v2.json); its execution against a running manager service has not been recorded as Newman run logs in the present writing, and the Postman collection is therefore presented as a structured demonstration artifact rather than as a recorded validation result. Recorded Postman execution is recognized as a hardening step planned alongside the PostgreSQL validation pass.

A continuous integration workflow is configured for the manager service at microservices/rfq_manager and runs an automated verification script on each push. The CI configuration is the only repository-level CI workflow in the project at present; CI coverage for the copilot, intelligence, and user interface services is recognized as a near-term hardening direction.

5.2.1 Endpoint Coverage and Response Validation

The pytest suite exercises the public API surface of the manager service across the seven resource families introduced in Chapter 3: RFQ, Workflow, RFQ_Stage, Subtask, Note, File, and Reminder. Test coverage spans the full set of endpoints exposed by these families: registration,

retrieval, stage advancement, attachment management, reminder configuration, and analytical aggregation. Each endpoint is exercised across its principal flows: the success path with valid input, the validation-failure path with malformed input, the not-found path for missing identifiers, and the authorization path for actor-context propagation.

The validation surface is structured to verify both response codes and response shapes. Successful responses are checked for the expected HTTP status code (200 for retrieval, 201 for creation, 204 for soft deletion where applicable), for the structural conformance of the response body to the published Pydantic schema, and for the presence of the operationally meaningful fields that downstream consumers depend on. Failure responses are checked for the expected HTTP status code (400 for malformed input, 403 for unauthorized access, 404 for missing resources) and for the structured error shape that the manager service returns, including the machine-readable error type and the correlation identifier described in Section 3.7 of Chapter 3.

The endpoint surface that the present implementation exposes evolved during the project from the V1 baseline of twenty-eight endpoints toward the broader operational surface of approximately thirty-one endpoints noted in Section 3.4 of Chapter 3, with the additional endpoints arising from hardening and from extensions to the reminder and statistics families. The pytest suite exercises the principal public API surface of the manager service across the seven resource families, with the response-validation coverage corresponding to the functional requirements FR-MGR-01 through FR-MGR-08 of the requirements catalogue in Chapter 2.

5.2.2 Workflow and Data Consistency

The lifecycle progression mechanics presented in Section 4.3.2 of Chapter 4 are exercised by a dedicated portion of the test suite that targets the three deterministic gates of the stage advancement controller: the mandatory-field gate, the blocker gate, and the transition-validity gate. Each gate is tested for its acceptance behaviour (the advance proceeds when the gate's conditions are satisfied) and for its rejection behaviour (the advance is refused with the appropriate structured error when the gate's conditions are not met). The atomicity of the commit step, in which a successful advance updates the stage state, the next-stage activation, and the lifecycle

timestamps within a single database transaction, is exercised through tests that introduce intermediate failures and verify that no partial advance leaves the lifecycle in an inconsistent state.

Data consistency at the supporting-resource level is exercised by tests that verify the soft-delete semantics of subtasks, the append-only contract of notes, the file-type discrimination at the upload endpoint, and the manual and rule-based creation modes of reminders. The progress recomputation that follows a subtask deletion is exercised through tests that verify the parent stage's progress denominator excludes soft-deleted subtasks, consistent with the requirement FR-MGR-08 in Chapter 2.

The traceability posture of the manager service, presented in Section 4.3.2 of Chapter 4, rests on lifecycle timestamps, persisted stage state, the append-only contract of notes, and the non-destructive update semantics of the supporting resources. The test suite verifies that each of these mechanisms operates as specified: timestamps are written on state changes, stage state transitions are persisted, notes cannot be modified or deleted through the public API, and soft deletion preserves the underlying record for audit purposes. The dedicated audit-history surface acknowledged in Chapter 4 is not exercised because it is not implemented in the present scope; its absence is consolidated in the limitations discussion of Section 5.8.

A summary of the manager service validation evidence is presented in Table 5.2.

Validation Activity	Status	Notes
Service-level pytest suite	259 passed, 0 failed	Run against SQLite quality-gate target.
Endpoint coverage across seven resource families	Implemented and tested	Covers FR-MGR-01 through FR-MGR-08.
Stage advancement gates (mandatory-field, blocker, transition-validity)	Implemented and tested	Acceptance and rejection paths exercised.
Soft-delete and append-only semantics	Implemented and tested	Subtasks, notes, files validated.
PostgreSQL-specific validation	Pending hardening	SQLAlchemy abstraction supports it; not in present evidence.
Postman walkthrough execution	Pending hardening	Collection exists; recorded run logs not in present evidence.
Continuous integration workflow	Configured for manager service	The only repository-level CI workflow at present writing.
Dedicated audit-history surface	Not implemented	Acknowledged hardening direction; see Section 5.8.

Table 5.2: Summary of validation evidence for the manager service

The manager service validation is therefore strong at the service level: the pytest suite covers the public API and the lifecycle mechanics, the architectural invariants of the controller layer hold against the test set, and the supporting resources are exercised consistently with

their declared semantics. The known gaps (PostgreSQL pass, recorded Postman execution, the audit-history surface) are presented as hardening directions rather than as gaps in the presently-claimed evidence.

5.3 Copilot Validation: Formal Metrics

The copilot service is the strongest validation area in the present work. Its pytest suite contains **636 tests across 47 test files**, all of which pass against the development environment used for the project’s validation runs. The full suite executes in under five seconds, reflecting the deterministic, test-double-backed structure of the test harness. A representative invocation and its outcome are summarized in Listing ??.

```
[caption=Copilot service pytest invocation and outcome., label=lst:copilottests, basicstyle =
, frame = single, breaklines = true] cd microservices/rfq_copilotms PYTHONPATH=. python
-m pytest -q -tb=no
```

```
636 passed, 3 warnings in 4.84s
```

The test suite is structured around three complementary categories, each owning a distinct class of evidence about the copilot’s behaviour. The first is **pipeline tests**, which exercise each individual stage of the turn processing pipeline introduced in Section 4.4 of Chapter 4 in isolation: FastIntake, the Planner, the PlannerValidator, the ExecutionPlanFactory, the Resolver, the Access stage, the Tool Executor, the Evidence Check, the Context Builder, the Compose stage, the Guardrails, the Judge, the Finalizer, the Escalation Gate, the Persist stage, and the Path 4 deterministic renderer. The pipeline test directory contains 216 test functions distributed across these stages. The second category is **smoke tests**, which exercise representative end-to-end flows of the pipeline and verify that the integrated behaviour of the stages produces the expected user-facing outcomes. The smoke test directory contains 98 test functions covering trivial conversational handling, RFQ-grounded answers, safe fallback under planner unavailability, ownership enforcement, and execution record persistence. The third category is **anti-drift tests**,

which verify the architectural invariants that anchor the trust-boundary architecture; these are presented in detail in Section 5.5 below.

The copilot test suite is qualified by two scope statements that are made explicit here. First, the language-model stages of the pipeline (the Planner, the Compose stage, and the Judge) are exercised through a deterministic test double called `FakeLlmConnector` rather than through live invocations of the configured Azure OpenAI deployment. The test double permits deterministic and reproducible verification of the pipeline's structural behaviour, which is what the trust-boundary architecture ultimately rests on, but does not validate the language-model response quality of the deployed model on production-scale traffic. Live-model regression validation is recognized as a future addition. Second, no continuous integration workflow is currently configured for the copilot service at the repository level; the test suite runs locally and on demand, with the addition of a CI workflow recognized alongside the broader CI hardening direction noted in Section 5.1.

A breakdown of the copilot test suite by category is presented in Table 5.3.

Test Category	Functions	Coverage
Pipeline tests	216	Per-stage verification of FastIntake, Planner, Validator, Factory, Resolver, Access, Tool Executor, Evidence Check, Context Builder, Compose, Guardrails, Judge, Finalizer, Escalation Gate, Persist, and the Path 4 deterministic renderer.
Smoke tests	98	End-to-end flows: trivial conversational handling, RFQ-grounded answers, safe fallback, ownership enforcement, persistence.
Anti-drift tests	18	Architectural invariants of the trust-boundary architecture; detailed in Section 5.5.
Other (fixtures, utilities, regression cases)	304	Supporting tests covering specific defects, configuration variants, and shared utilities.
Total	636	All passing in the validation runs of the project.

Table 5.3: Breakdown of the copilot service pytest suite by category

The 636 passing tests therefore validate the copilot service’s structural behaviour comprehensively at the service level. The two qualifications stated above (deterministic test double for the language model, no automated CI workflow) are the only material qualifications that apply

to the test evidence; with those qualifications acknowledged, the validation is the strongest in the project.

5.4 Copilot Validation: Golden Scenarios

The validation methodology adopted for the copilot does not include a separately-curated benchmark dataset of golden user queries with expected answers, of the kind that would be appropriate for an evaluation pass against the deployed language model. The construction of such a dataset is recognized as a future direction, and is discussed alongside live-model regression validation in Section 5.9. The present work instead validates the copilot’s expected behaviour through smoke and pipeline tests that function as automated golden scenarios: each test fixes an input, exercises the pipeline end-to-end (or a defined subset of it), and asserts the expected user-facing outcome.

The scenarios that the smoke and pipeline tests collectively exercise span the behavioural surface of the first vertical slice presented in Section 4.4.1 of Chapter 4. **FastIntake-classified trivial messages** are exercised through tests that submit greetings, thanks, farewells, and pure-nonsense inputs and verify that the appropriate Path 1 or Path 8.2 response is returned without invoking the Planner or the language model. **RFQ-grounded Path 4 flows** are exercised through tests that submit operational queries about a specific RFQ (status questions, attached-artifact questions, lifecycle progression questions) and verify that the response is grounded in evidence retrieved from a fake manager connector with the expected per-target labelling and field whitelisting. **By-code RFQ lookup** is exercised through tests that submit RFQ references by their code rather than by their internal identifier and verify that the resolver and the access stage handle the lookup correctly. **Manager-core Path 4 flow** is exercised through tests that drive the full pipeline from intake through persistence with manager-resident evidence and verify that the execution record captures the per-stage outcomes accurately. **Guardrail behaviour** is exercised through tests that introduce drafts that violate the evidence guardrail, the scope guardrail, or the shape guardrail, and verify that the offending draft is intercepted and routed through the Escalation Gate to the appropriate Path 8 finalization. **Path 8 safe fallback** is exercised through

tests that introduce planner unavailability, evidence emptiness, access denial, target ambiguity, and out-of-scope classification, and verify that each failure mode produces the appropriate Path 8 sub-case with the correct reason code. **Ownership enforcement** is exercised through tests that submit cross-thread access attempts and verify that the copilot enforces the thread-owner restriction. **Known-thread requirement** is exercised through tests that submit turns referencing a non-existent thread and verify that the request is rejected before any downstream stage runs. **Execution record persistence** is exercised through tests that complete a turn end-to-end and verify that the persisted execution record contains the expected per-stage decisions, evidence references, and outcome metadata.

These scenarios collectively exercise the behavioural commitments of the first vertical slice as established in Chapter 4. The copilot is not validated against an external benchmark dataset of natural-language queries with curated expected answers; it is validated against a structured suite of behavioural scenarios that codify the expected pipeline-level outcomes. The distinction between automated scenario coverage and benchmark dataset evaluation is preserved here as a scope statement: scenario coverage demonstrates that the architecture behaves as designed; benchmark dataset evaluation would additionally demonstrate that the deployed language model produces high-quality natural-language responses across a representative distribution of real user queries. The latter is future work.

5.5 Integration Validation

The four implemented services of the Itqan platform compose into a coherent end-to-end experience through the integration paths presented in Section 3.8 of Chapter 3. The validation of these integration paths is more partial than the service-level validation presented in Sections 5.2 through 5.5, and this section reports the integration evidence with the scope honesty that the foundation-level maturity of the user interface and the intelligence service requires.

5.5.1 Integration Paths and Validation Status

Three principal integration paths govern the cross-service behaviour of the platform. The first is the **user interface to backend path**, in which the user interface consumes the manager service's lifecycle and analytical endpoints, the intelligence service's parsed artifacts, and the copilot service's conversational layer. The second is the **copilot to evidence path**, in which the copilot service retrieves operational evidence from the manager service to ground a Path 4 answer. The third is the **manager to intelligence path**, in which an estimation workbook uploaded against an RFQ in the manager service is parsed by the intelligence service and its profile retrieved by downstream consumers.

The validation evidence for each path is structured to acknowledge what is exercised in tests, what is exercised through manual demonstration, and what remains as a hardening direction. Table 5.4 summarizes the validation status of the principal integration paths.

Integration Path	Status	Validation Evidence
UI to manager service	Demonstration	Source-contract scripts and manual demonstration walkthroughs; no browser-driven end-to-end tests in the present scope.
UI to copilot service	Demonstration	Manual demonstration of the RFQ-bound and portfolio-level conversational entry points; thread management currently consumes the v1 lane while turn submission consumes the v2 lane.
UI to intelligence service	Demonstration	Read-only consumption of parsed artifacts surfaced through the intelligence panel of the RFQ detail view.
Copilot to manager service	Tested via test double	Exercised through smoke and pipeline tests using a deterministic manager connector double; live manager integration validated through manual scenario runs.
Manager to intelligence service	Manual/direct trigger	Workbook parsing initiated through manual or direct trigger paths; autonomous event-bus consumer architecturally specified but not wired.
Copilot to intelligence service	Not implemented	Connector for runtime consumption of intelligence artifacts deferred to the slices that ship Path 5 and related
ENET'COM SFAX		intelligence-driven paths. Page 121

Table 5.4: Integration paths and their validation status in the present work

5.5.2 Demonstration-Oriented Evidence

The integration evidence assembled in the present work is demonstration-oriented rather than test-automated, and the artifacts that support this evidence are concrete and reproducible even in the absence of browser-driven tests. A scenario stack script, located at `scripts/rfqmgmt_scenario_s` in the project repository, brings up a representative multi-service configuration suitable for end-to-end demonstration. A smoke demonstration document at `docs/SMOKE_DEMO.md` provides the step-by-step walkthrough that exercises the platform from user interface through to the copilot's evidence-grounded response, including the path 4 demonstration that anchors the project narrative. Seed manifests under `seed_outputs` populate the manager service with representative RFQ data so that the demonstration is exercised against realistic lifecycle content rather than against empty fixtures.

The frontend itself is validated through a set of source-contract scripts that exercise the integration boundaries of the user interface against expected response shapes. Twelve such scripts are present in the codebase, and eleven of them pass against the audited branch; the single failing script reflects assertion drift rather than a runtime regression and is acknowledged here as a maintenance item rather than as a validation gap. Browser-level end-to-end testing of the user interface, through tools such as Playwright, Cypress, or React Testing Library, is not present in the current scope; testing of the integrated platform is currently performed through the combination of the manager service's pytest suite, the copilot service's pytest suite, the intelligence service's fixture-independent tests, the source-contract scripts, and the manual demonstration walkthroughs supported by the scenario stack and smoke demonstration document.

5.5.3 Intelligence Service Validation in the Integrated Context

The intelligence service occupies a distinctive position in the integration validation. Its own pytest suite contains **118 tests across 39 test files**, of which 64 fixture-independent tests pass against the audited branch and 6 tests are skipped by design. The remaining 48 tests are fixture-dependent: they are designed to validate the parsers against the golden reference artifacts

introduced in Chapter 1 (the SA-AYPP-6-MR-022 package and the IF-25144 workbook), and they fail with `FileNotFoundError` when those artifacts are not present in the local fixture directory. In the audited checkout, this fixture-dependent validation could not be reproduced because the fixture directory was absent; the failures therefore reflect the environmental absence of the external artifacts rather than parser regressions. The fixture-independent tests covering briefing, intake, and snapshot logic pass without external dependencies.

The intelligence service's contribution to the integration story is therefore that the supporting briefing, intake, and snapshot logic is validated independently of external artifacts, and the integration into the manager service is exercised through the manual and direct trigger paths described above. Reproducible parser validation against the golden artifacts in a clean checkout is acknowledged as a hardening direction: making the fixtures available outside the gitignored local-fixtures directory would let the full intelligence test suite be exercised when CI coverage is extended to the intelligence service.

5.5.4 Summary of Integration Validation

The integration validation of the Itqan platform is therefore partial and demonstration-oriented, in the deliberate sense established at the outset of this section. Service connectors and scenario scripts exist, copilot-to-manager behaviour is validated through deterministic test doubles in the copilot's pytest suite and through manual scenario runs against the live manager service, and the platform is prepared for structured manual demonstration through the scenario stack, seed manifests, and smoke demonstration document. These artifacts support reproducible demonstration of the main flows, while recorded stakeholder walkthrough evidence and automated full-stack end-to-end logs remain future validation hardening items. Browser-driven full-stack end-to-end tests, autonomous event-driven manager-intelligence cascade, and live manager-copilot integration tests at the automated test level are recognized as the next layer of integration hardening; their absence does not invalidate the integration evidence presented here, but it bounds the strength of that evidence to the demonstration level rather than the production-grade automated level.

The next section turns from validation evidence to the business impact of the implemented capabilities, mapping them back to the audit pain points identified in Chapter 1.

5.6 Business Impact Against Audit Pain Points

The validation activities of Sections 5.2 through 5.6 establish that the Itqan platform behaves as designed. This section turns from technical validation to business impact: it asks what the implemented capabilities do for the operational reality observed during the audit, and it answers by composing the twelve audit pain points presented in Section 1.4 of Chapter 1 with the implementation evidence assembled across the rest of the report. The mapping is structured deliberately to distinguish pain points that the platform addresses substantively from those it addresses partially and from those that remain in scope but are not yet impacted by the present work.

Mapping Methodology

Each of the twelve pain points identified during the audit is examined in turn against the capabilities the platform now provides. For each pain point, three judgments are made. The first identifies the capabilities the platform implements that bear on the pain point. The second classifies the degree to which the pain point is addressed in the present work, using a three-level vocabulary: *addressed* when the implemented capabilities meet the pain point’s structural cause, *partially addressed* when the implemented capabilities reduce but do not eliminate the pain point, and *not yet addressed* when the pain point lies beyond the present scope despite being recognized in the platform’s vision. The third judgment identifies the architectural mechanism through which the pain point is addressed, so that the impact claim can be verified against the implementation rather than accepted on its face.

The mapping is presented in Table 5.5.

ID	Pain Point	Status	Implementing Mechanism
PP-01	Inter-departmental communication breakdown	Addressed	Manager service lifecycle representation, stage progression, and shared artifact attachment provide a single coordinated view across departments.
PP-02	Manual client follow-up	Partially addressed	Reminder infrastructure attached to lifecycle artifacts is implemented; outbound channel delivery is log-only and the unified inbound monitoring of Pillar 2 is future scope.
PP-03	Zero executive visibility	Partially addressed	Pipeline, win-rate, and by-client analytical endpoints are implemented and surfaced through the user interface dashboard; full BI surface including profitability is future scope.
PP-04	No reuse of past RFQs	Not yet addressed	Lifecycle data is now structured and persisted, providing the foundation for future historical learning, but the predictive layer that would exploit this data is deferred.
PP-05	Late discovery of infeasibility	Partially addressed	Stage-level blocker fields and the blocker gate of the stage advancement controller surface infeasibility earlier; deeper feasibility analytics are future scope.
PP-06	Workbook as single point of failure	Partially addressed	Workbook upload and structured parsing through the intelligence service produce a deterministic profile that complements the workbook itself; the workbook remains the authoritative pricing instrument.
PP-07	Undocumented feasibility expertise	Partially addressed	Append-only notes and structured attachment of artifacts to lifecycle stages capture feasibility decisions and reasoning in a queryable form; structured prompts to encode tacit knowledge are future scope.
ENET'COM SFAX			

Reading the Mapping

Three observations are worth surfacing from the mapping. The first is that two pain points (PP-01 and PP-08) are substantively addressed by the operational backbone alone: the manager service's lifecycle representation and its workflow-driven stage progression directly address the structural causes of the inter-departmental communication breakdown and of the classification-to-workflow gap, because both pain points were caused by the absence of a shared, coordinated representation that the manager service now provides. The platform delivers the structural mechanism needed to reduce these causes; the realization of that reduction in operational practice depends on adoption activities that lie beyond the present technical scope. These are the platform's strongest direct contributions.

The second observation is that seven pain points are partially addressed. In each case, the platform makes the underlying problem visible, queryable, and structurally tractable, but the full elimination of the pain point depends on capabilities that lie in future scope: outbound communication channels, deeper analytical surfaces, predictive layers, and standards-driven cost engines. Partial addressing is not a weakness here; it is the consequence of the strategic pivot presented in Section 1.5 of Chapter 1, which prioritized the platform foundation that everything else will be built on, rather than attempting a thinner attempt at every pillar simultaneously.

The third observation is that three pain points are not yet addressed, and each of these falls clearly within the future scope established by the four-pillar vision: PP-04 (historical learning) and PP-09 (supplier intelligence) belong to Pillar 4, and PP-10 (ERP utilization) belongs to a future integration direction that lies beyond the four pillars proper. None of these is claimed as addressed in the present work, and their explicit acknowledgment here closes the loop between the original audit findings and the present state of the platform.

The platform therefore delivers measurable progress against the audit findings of Chapter 1: two pain points substantively addressed, seven partially addressed through structurally enabling mechanisms, and three explicitly deferred within a known future scope. The next section discusses the limits of this work in honest terms.

5.7 Limitations and Honest Discussion

The validation evidence assembled in this chapter is the strongest argument the report can make for the work's claims, but every project has limits, and an honest report names them. This section consolidates the limits of the present work in one place. Each limit is presented with the reason it exists, the consequence it has for the work's defensibility, and the path through which it would be addressed.

Validation-Methodology Limits

The validation methodology rests on automated tests at the service level rather than on browser-driven end-to-end tests, on a deterministic test double rather than on live language-model invocations for the copilot's structural validation, and on per-service test execution rather than on a continuous integration workflow that exercises every service on every commit. Each of these choices is defensible in the context of a solo end-of-studies project with finite time, but each one bounds the strength of the evidence the report can claim. Browser-driven end-to-end tests would let the integration validation reach automated rather than demonstration-oriented status. Live-model regression tests would extend the trust-boundary verification to include adversarial probing of the deployed Azure OpenAI model. Repository-level CI for every service would prevent regressions of the kind that the manager service's CI workflow already catches. The present work claims none of these as delivered; each is recognized as the next layer of validation hardening.

Implementation-Surface Limits

Several capabilities described by the platform's vision are implemented at foundation level rather than at the level of the long-term scope. The intelligence service ships a parsing foundation validated against a single golden reference package and a single golden reference workbook; broader validation against additional artifacts is recognized as a hardening direction. The user interface ships demonstration-grade screens that exercise the integration surface but do

not include browser-level testing or production login flow; the actor context that the frontend submits is selected client-side through a development-mode role selector. Frontend automated validation in the present scope is limited to source-contract scripts that exercise the integration boundaries against expected response shapes; it does not verify rendered UI behaviour in a browser, and the addition of browser-level end-to-end tests through tools such as Playwright or Cypress is recognized as a hardening direction. The dedicated identity and access management service is structurally a placeholder, with backend authentication mechanisms in place and a dedicated service deferred. The autonomous event-bus consumer that would let the manager-to-intelligence cascade run without explicit invocation is architecturally specified and not wired. The copilot's connector for runtime consumption of intelligence artifacts is not implemented in the present work.

Architecture-Scope Limits

The trust-bound copilot architecture, the principal innovation contribution of the work, is implemented for the first vertical slice only. Path 1 (trivial conversational handling), Path 4 (the demo-defining operational path), and the full Path 8 family (the safety infrastructure) ship together, while the remaining answer paths (2, 3, 5, 6, 7) route safely through the Escalation Gate to Path 8.1 until their slices ship. This coverage strategy was deliberate: the audit finding from the architecture review, namely that shipping Path 4 and Path 8 together exercises every component of the pipeline end-to-end on a real operational path, is the property that makes the slice-by-slice expansion of path coverage tractable. The limit is therefore that a substantial portion of the architecturally specified answer space is not yet exercised against real user queries; that exercise is the work of subsequent slices.

Working Memory and Conversation Continuity

Working memory in the copilot's present implementation is captured but not yet injected into the language-model invocations. The conversation history is structured, queryable, and forensically complete; what it is not yet is part of the context that shapes the next turn's

answer. This is a deliberate scope discipline that prevents under-validated memory injection from compromising the trust-boundary architecture, but it is also a limit that the present work acknowledges: the copilot's behaviour on a given turn is determined by that turn's content, not by accumulated conversational context. Memory injection and long-term episodic summarization are recognized as future extensions to the memory architecture.

Honest Statement of Risk

The risks tracked through the project's risk register, presented in Chapter 2, materialized to varying degrees during the work. Scope creep beyond the implemented baseline was managed through the strategic pivot presented in Chapter 1 and the slice-by-slice path coverage strategy presented above. Dependence on third-party API stability for the language-model layer of the copilot is mitigated by the deployment-environment indirection but remains a structural exposure. The limited scale of the validation dataset, particularly the absence of a curated golden benchmark for natural-language answer quality, is the most consequential validation gap and is named explicitly here as the highest-priority hardening item for any subsequent work. The academic timeline pressure relative to the platform's full vision is what produced the deliberate scoping decisions documented throughout the report; it is not a regret but a constraint that shaped the work's identity.

5.8 Future Work

The future work directions that follow from the limits enumerated above can be organized into three priority tiers that reflect both the urgency of each direction and the dependencies between them.

Near-Term: Validation Hardening

The most immediate future work is the validation hardening that would convert the demonstration-oriented integration evidence of the present work into automated production-grade evidence. This includes

the introduction of a continuous integration workflow that exercises the copilot, intelligence, and frontend services on every commit, the extension of the manager service's pytest suite to a PostgreSQL execution mode, the recording of Postman collection runs against a deployed manager service, the construction of a curated golden benchmark dataset for evaluating the deployed language model's answer quality across a representative distribution of user queries, and the introduction of browser-level end-to-end tests for the user interface. Each of these is a hardening direction rather than a new capability; together they would convert the validation chapter of a future report into a substantially stronger document than the present one.

Medium-Term: Architecture Completion

The medium-term future work centres on completing the architectural surface that the present work has delivered as foundation. The remaining copilot answer paths (2, 3, 5, 6, 7) would ship slice by slice, each exercising the full pipeline against the path's specific evidence sources and policy constraints. The autonomous event-bus consumer that would let the manager-to-intelligence cascade run without explicit invocation would be wired, eliminating the manual trigger paths that bridge the two services in the present implementation. The copilot's connector for runtime consumption of intelligence artifacts would be implemented, completing the cross-service evidence path that Section 3.5 of Chapter 3 establishes architecturally. The dedicated identity and access management service would be extracted from the in-service backend authentication shim, delivering the production login flow and the SSO integration that the user interface currently lacks. The working-memory injection into the language-model invocations would be implemented under the trust-boundary discipline, with the appropriate guardrail extensions and validation that the architecture's properties hold under the new memory inputs.

Long-Term: Pillar 4 and Predictive Intelligence

The long-term future work is the predictive intelligence layer of Pillar 4 that the four-pillar vision establishes as the deepest extension of the platform. This includes pattern matching against historical RFQs, preliminary cost-range guidance derived from comparable bids, supplier

intelligence built from cumulative procurement history, and win-probability estimation conditioned on client and project characteristics. Each of these capabilities depends on a structured historical dataset that the manager service is now beginning to accumulate; the predictive layer becomes feasible once the dataset reaches the volume and structural maturity that learning-based methods require. The long-term direction therefore unfolds naturally from the foundation that the present work has put in place: the manager service's structured lifecycle representation is what the predictive layer would learn from, and the trust-bound copilot architecture is what would deliver the learned insights to users under the same evidence and authority discipline that the present work establishes.

The platform that the present work has built is the substrate for everything above. Each tier of future work extends the platform without reopening the architectural commitments that anchor it; the eight trust boundaries, the data ownership posture, the layered structure of each service, and the policy-as-configuration discipline of the Path Registry remain the foundation on which subsequent work would build.

5.9 CONCLUSION

This chapter has validated the implementation of the Itqan platform against the requirements catalogued in Chapter 2 and the architectural commitments established in Chapter 3. It presented the manager service's pytest suite of 259 passing tests as evidence for the operational backbone, the copilot service's pytest suite of 636 passing tests as evidence for the trust-bound conversational layer, the eighteen anti-drift tests as the verification surface for the eight trust boundaries that distinguish the copilot's architecture from a generic conversational system, and the integration evidence drawn from scenario stack scripts, smoke demonstrations, and source-contract scripts that compose the four implemented services into a coherent platform. It mapped the implemented capabilities against the twelve audit pain points identified in Chapter 1, finding two pain points substantively addressed by the operational backbone alone, seven partially addressed through structurally enabling mechanisms, and three explicitly deferred within the future scope of the platform's four-pillar vision. It enumerated the limits of the present work honestly, in five

categories spanning validation methodology, implementation surface, architectural scope, memory and conversation continuity, and project-level risk. It set out the future work that follows from these limits in three priority tiers, ordered by urgency and by the dependencies between them.

The platform that this report describes is the foundation of a longer trajectory. Within the scope of the present work, it delivers an operational backbone whose lifecycle representation directly addresses the structural causes of the most consequential audit pain points, a conversational layer whose trust-boundary architecture enforces the distinction between language and authority that an industrial estimation context requires, an intelligence parsing foundation that produces structured profiles of the documents the lifecycle revolves around, and an integration surface that composes these services into an experience that operationalizes the vision presented in Chapter 1. The report as a whole therefore concludes that the strategic pivot from BOQ automation to RFQ lifecycle intelligence, justified in Chapter 1, defended architecturally in Chapter 3, implemented under disciplined scope in Chapter 4, and validated through the evidence assembled in this chapter, has produced a platform whose foundations are sound, whose innovation is verifiable, and whose direction is established. The General Conclusion that follows returns to the project as a whole and reflects on its contribution.



GENERAL CONCLUSION

[Placeholder] This concluding chapter synthesizes the contributions of the work and reflects on what was achieved, why it matters, and what comes next.

[Placeholder — paragraph one: what was achieved] This paragraph will summarize the concrete deliverables of the project: a working RFQ lifecycle management service, a trust-bound conversational copilot grounded in platform evidence, an integrated platform that exposes both engines through a coherent user experience, and a validation strategy that combines backend rigor with formal copilot metrics and golden scenario testing.

[Placeholder — paragraph two: why it matters] This paragraph will articulate the business and engineering significance of the work, situating Itqan against the audit pain points identified at the client site and against the broader challenge of building trustworthy AI systems for industrial estimation contexts.

[Placeholder — paragraph three: technical skills and engineering judgment demonstrated] This paragraph will identify the technical competencies exercised during the project: microservices design, the FastAPI and SQLAlchemy stack, LLM integration with epistemic guardrails, the BACAB layered pattern, structured project management with Scrumban and ADR practices, and the discipline of strategic reframing when the original problem definition proved infeasible.

[Placeholder — paragraph four: innovation contribution] This paragraph will name the central innovation contribution of the work: the trust-boundary architecture as a defensible pattern for constrained agentic systems, separating language understanding (owned by the LLM) from operational truth, access control, evidence selection, and response boundaries (owned by deterministic platform components).

[Placeholder — paragraph five: personal perspective and future directions] This paragraph will close on a personal reflection: what the project taught about engineering judgment in the face of ambiguity, what remains to be built across the four-pillar vision, and how the platform might evolve toward multi-client deployment and federated learning across RFQ histories.



BIBLIOGRAPHY

- [1] **ENET’Com**. National School of Electronics and Telecommunications of Sfax **[online]**.
Available at: <https://enetcom.rnu.tn/>
- [2] **Leslie Lamport**. (1994) *TEX: a document preparation system*, Addison Wesley, Massachusetts, 2nd ed.
- [3] [Placeholder] Reference to be added in Chapter 3 covering the principles of grounding large language model responses in retrieved or structured evidence.
- [4] [Placeholder] Reference to be added in Chapter 3 covering retrieval-augmented generation as a baseline pattern that the trust-boundary architecture extends.
- [5] [Placeholder] Reference to be added in Chapter 3 covering tool-using LLM agents and the failure modes that motivate constrained agentic designs.
- [6] [Placeholder] Reference to be added in Chapter 4 covering the FastAPI framework documentation as the basis for the manager and copilot HTTP layer.
- [7] [Placeholder] Reference to be added in Chapter 4 covering SQLAlchemy as the persistence layer and Alembic for migrations.
- [8] [Placeholder] Reference to be added in Chapter 3 covering microservice architectural principles, service boundaries, and the layered patterns that informed the BACAB approach.
- [9] [Placeholder] Reference to be added in Chapter 2 covering Scrumban as a hybrid of Scrum cadence and Kanban work-in-progress discipline.

- [10] [Placeholder] Reference to be added in Chapter 1 covering applicable ASME standards for pressure vessel design referenced in the RFQ domain.
- [11] [Placeholder] Reference to be added in Chapter 1 covering applicable TEMA standards for heat exchanger classification referenced in the RFQ domain.



APPENDICES

A.1 Detailed Use Case Descriptions

[Placeholder] This appendix provides extended use case descriptions complementing the use case diagram presented in Chapter 2. Each use case is documented with its actor, preconditions, main flow, alternative flows, and postconditions.

A.2 Additional Sequence Diagrams

[Placeholder] This appendix presents the sequence diagrams that complement the architecture chapter, covering interactions that are referenced in the body but rendered in detail here for the interested reader.

A.3 Selected Path Registry Excerpt

[Placeholder] This appendix presents a selected excerpt of the Path Registry configuration used by `rfq_copilot_ms`, illustrating the structure of a path entry: allowed fields, memory policy, judge triggers, finalizer template, and model profile.

A.4 Copilot Prompt Excerpt and Response Rules

[Placeholder] This appendix presents a representative excerpt of the copilot system prompt, including the identity and persona section, the chain-of-thought instruction, scope and grounding rules, and a subset of the few-shot examples that anchor the model's behavior.

A.5 Golden Scenario Test Set

[Placeholder] This appendix presents the full golden scenario test set used to validate the copilot, including the scenario identifier, the input message, the expected behavior, the expected reason code, and the observed result. The summary results are reported in Chapter 5.

A.6 API and Postman Validation Structure

[Placeholder] This appendix presents the structure of the Postman collection used to validate `rfq_manager_ms`, including the request groups, the test assertions per request, and the environment variables used to drive the collection.

PROJECT TITLE

First name Last name

Résumé:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue.

Mots clés: Lorem, ipsum, dolor, sit amet, consectetur, adipiscing elit.

Abstract :

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue.

Key-words: Lorem, ipsum, dolor, sit amet, consectetur, adipiscing elit.

تلخيص:

هذا التقرير يصف

الكلمات المفتاحية: